

Escuela Politécnica Superior

Nombre del alumno: Gabriel Cárdena López

Nombre/s de los tutores profesional y académico:

Alonso Martín Toledano García Mauriño

David Tuñón Cabeza

Tipología de memoria: Investigación

Evaluación del impacto de los Modelos de Lenguaje en la Optimización de Consultas SQL para Diferentes Motores de Procesamiento de Datos

**Evaluación de Capacidades
en la Empresa I**

Grado en ingeniería informática



UNIVERSIDAD
NEBRIJA

D. /Dña. Gabriel Cárdbaba López autoriza a que el presente trabajo se guarde y custodie en los repositorios de la Universidad Nebrija y además NO autoriza a su disposición en abierto.

Índice

Índice	4
Glosario de términos	7
Índice de ilustraciones	9
Índice de tablas	11
1. Resumen	13
2. Prácticas	14
2.1. Antecedentes	14
2.1.1. Entrevistas realizadas	14
2.1.2. Información previa	14
2.1.3. Motivos de elección de prácticas	15
2.1.4. Datos de la empresa	15
2.2. Objetivos de las prácticas	16
2.3. Onboarding	17
2.4. Funciones y tareas en las prácticas.	18
2.5. Herramientas utilizadas en las prácticas (evidencias)	18
2.6. Metodologías utilizadas	19
2.7. Valoración personal de la elección de las prácticas	19
3. Investigación/Desarrollo.....	21
3.1. Introducción	21
3.2. Antecedentes	23
3.3. Estado del arte	25
3.4. Necesidad detectada	29
3.5. Objetivos	30
4. Marco teórico.....	32
4.1. Modelo relacional y procesamiento de consultas SQL	32
4.2. Optimización de consultas basada en costes	32
4.3. Estimación de cardinalidad	32
4.4. Motores de procesamiento de datos: PostgreSQL y Apache Spark	33
4.5. Modelos de Lenguaje (LLMs) aplicados a la reescritura de consultas	33
4.6. El benchmark TPC-H	33
5. Metodología.....	35
Fase 4: Evaluación y Medición	38
6. Proyecto.....	39
6.1. Resumen de contribuciones y productos desarrollados	39
6.2. Planificación temporal	40
6.3. Recursos empleados	40
6.4. Valoración económica	41

7. Resultados y discusión.....	43
8. Conclusiones	53
9. Líneas futuras	55
10. Uso de la IA.....	56
Bibliografía	57
Anexos (Opcional)	59
Anexo A. Repositorio del proyecto	¡Error! Marcador no definido.
Anexo B. Script de conversión a Parquet (ConvertToParquet.scala)	59

Glosario de términos

SQL (Structured Query Language): Lenguaje de programación estandarizado para gestionar bases de datos relacionales.

ETL (Extract, Transform, Load): Proceso de extracción, transformación y carga de datos desde fuentes origen hacia un sistema destino, típicamente un data warehouse o data lake.

Data Lake: Repositorio centralizado que permite almacenar grandes volúmenes de datos estructurados y no estructurados.

LLM (Large Language Model): Modelo de Lenguaje de Gran Escala, sistema de inteligencia artificial entrenado con grandes cantidades de texto para generar, comprender y transformar lenguaje natural y código.

PostgreSQL: Sistema de gestión de bases de datos relacional de código abierto, soporta SQL estándar.

Apache Spark: Motor de procesamiento de datos distribuido diseñado para análisis de grandes volúmenes de datos (Big Data) con capacidades de procesamiento en memoria.

QA (Quality Assurance): Proceso de verificación de calidad que garantiza la integridad, consistencia y corrección de los datos y sistemas.

Batch: Procesamiento por lotes, método de ejecución de tareas que procesa conjuntos de datos completos en intervalos programados.

Streaming: Procesamiento de datos en tiempo real o casi real.

Query/Consulta: Instrucción SQL que solicita datos específicos de una base de datos mediante operaciones de selección, filtrado, agregación o transformación.

Azure (Microsoft Azure): Plataforma de servicios en la nube de Microsoft que ofrece infraestructura, almacenamiento, procesamiento y servicios de datos.

Scala: Lenguaje de programación multi-paradigma que combina orientación a objetos y programación funcional, utilizado frecuentemente con Apache Spark.

Shuffle: Operación costosa en sistemas distribuidos que redistribuye datos entre nodos, típicamente necesaria para joins, agregaciones y ordenaciones.

Join: Operación que combina filas de dos o más tablas basándose en una condición relacionada entre columnas.

Buffer: Área de memoria temporal utilizada para almacenar datos durante operaciones de lectura/escritura, reduciendo accesos a disco.

I/O (Input/Output): Entrada/salida de información.

Pipeline: Secuencia de procesos de transformación de datos conectados donde la salida de uno es la entrada del siguiente.

BI (Business Intelligence): Conjunto de estrategias, aplicaciones y tecnologías para análisis de datos empresariales.

Índice de ilustraciones

Ilustración 1. Horario	19
Ilustración 2. Query 1 Desviación Típica PostgreSQL	45
Ilustración 3. Query 1 Mejora Porcentual en Tiempo de Ejecución de Consultas SQL en PostgreSQL por LLM.....	46
Ilustración 4. Query 1 Comparación de tiempo medio y desviación típica de ejecución PostgreSQL vs SPARK	46
Ilustración 5. Query 1 Desviación Típica SPARK	47
Ilustración 6. Query 1 Mejora Porcentual en Tiempo de Ejecución de Consultas SQL en SPARK por LLM	47
Ilustración 7. Query 2 Desviación Típica PostgreSQL	49
Ilustración 8. Query 2 Mejora Porcentual en Tiempo de Ejecución de Consultas SQL en PostgreSQL por LLM.....	50
Ilustración 9. Query 2 Comparación de tiempo medio y desviación típica de ejecución PostgreSQL vs SPARK	50
Ilustración 10. Query 2 Desviación Típica SPARK	51
Ilustración 11. Query 2 Mejora Porcentual en Tiempo de Ejecución de Consultas SQL en SPARK por LLM	52

Índice de tablas

Tabla 1. Objetivos de las prácticas y grado de consecución.	17
Tabla 2. Planificación temporal del proyecto.	40

1. Resumen

Durante mi periodo de prácticas en VML - The Cocktail, desde el 13 de octubre de 2025 hasta el 24 de mayo de 2026, formé parte del departamento de Data & Marketing desempeñando el rol de Data Engineer. Mis responsabilidades incluyeron tareas de análisis de datos, optimización de consultas automatizadas tanto en batch como en streaming, análisis de QA, mantenimiento de la arquitectura de datos del proyecto asignado, un data lake denominado MDL (Media-Data-Lake) y el desarrollo y mejora de procesos ETL. Todo ello lo llevé a cabo en colaboración con un equipo técnico compuesto por un arquitecto, un Senior Data Engineer, varios perfiles de BI, el Project Manager de la parte técnica y yo como becario de Data Engineering, además de un equipo funcional y una persona dedicada exclusivamente a QA.

El auge de los motores de procesamiento de datos (PostgreSQL, Apache Spark) ha incrementado la complejidad de la optimización de consultas SQL, donde el coste y el rendimiento dependen de particularidades del optimizador y el formato de datos entre otras cosas. Los Modelos de Lenguaje (LLMs) se posicionan como asistentes potenciales para sugerir reescrituras de consultas adaptadas al motor. El objetivo de este proyecto es evaluar si distintos LLMs (GPT-5, Gemini 2.5, Claude Sonnet 4.5) pueden proponer reescrituras SQL que mejoren rendimiento manteniendo corrección semántica, y comparar su impacto entre motores. Se tomarán consultas representativas desarrolladas en un servidor, se ejecutarán versiones base y reescritas por LLMs, verificando equivalencia de resultados y midiendo tiempo, coste, I/O y plan de ejecución. Se analizarán patrones de optimización por motor (proyecciones tempranas, filtros empujados, orden de joins, uso de agregaciones parciales, hints). Resultados principales: Se reportarán mejoras o degradaciones por motor, robustez de las reescrituras, y sensibilidad a prompts. Se identificarán clases de consultas donde los LLMs aportan valor consistente. El estudio pretende delimitar cuándo y cómo integrar LLMs en la fase de optimización de consultas SQL, proponiendo guías prácticas por motor y una infraestructura replicable de evaluación.

2. Prácticas

2.1. Antecedentes

En el momento de comenzar las signaturas de EVALUACIÓN DEL DESARROLLO DE CAPACIDADES EN LA EMPRESA I y II tengo pendiente 3 asignaturas de tercer curso, durante el primer cuatrimestre escolar estoy cursando la asignatura de Sistemas empotrados y durante el segundo cuatrimestre estaré cursando IA y PROGRAMACIÓN DE INTERFACES WEB, con la idea de desarrollar y presentar en TFG el curso siguiente (2026-2027) para liberar la carga de trabaja este curso escolar y poder atender a todas las responsabilidades con el tiempo y esfuerzo necesario.

2.1.1. Entrevistas realizadas

A lo largo de mi proceso de búsqueda de prácticas, mantuve contacto con varias empresas. En el caso de EY, el procedimiento se inició mediante una llamada telefónica con intención de continuar con el proceso de selección. No obstante, tras este primer contacto, no recibí comunicación adicional por parte de la compañía, lo que dejó el proceso inconcluso.

Con Management Solutions, el desarrollo del proceso de selección fue más extenso. Comencé con una primera entrevista con el departamento de Recursos Humanos, seguida de una segunda entrevista con un profesional del ámbito de la informática. Posteriormente, mantuve una tercera reunión con uno de los gerentes de la firma. Sin embargo, al pasar a la que debía ser la cuarta y última fase, no obtuve respuesta hasta un mes después, retraso que generó una considerable falta de claridad respecto a la continuidad del proceso.

En VML - The Cocktail, todo fue más ágil y ordenado. Inicialmente, tuve una breve llamada con el proposito de verificar los datos rellenos en la solicitud. Mas tarde, se llevó a cabo una entrevista online con el departamento de Recursos Humanos y, finalmente, una última entrevista con un Data Engineer de la empresa, quien posteriormente resultaría ser el Senior asignado a mi proyecto.

2.1.2. Información previa

Al ser un puesto de Data engineer las asignaturas que más me están ayudando en el desempeño de mi puesto son Bases de Datos para todo, desde entender la arquitectura

utilizada en el proyecto, entender los requerimientos de clientes y los internos, hasta hacer queries para un análisis de QA más amplio e ir al directamente al dato. Asimismo, otra asignatura que me ha ayudado en el día a día es ARQUITECTURA Y PROGRAMACIÓN DE SISTEMAS EN INTERNET donde aprendimos a desarrollar API's en TypeScript, ya que hacemos uso de API's para automatizar tareas y desarrollamos herramientas en JavaScript y HTML para facilitar tareas al equipo funcional. Una asignatura que me ha ayudado en un grado menor, pero aporta unas bases frente a otros compañeros de carrera universitaria y laboral es la asignatura CLOUD COMPUTING. HERRAMIENTAS Y APLICACIONES del Diploma opcional de la carrera de Ingeniería Informática ya que aprendimos sobre AWS y en el trabajo utilizamos la infraestructura de Microsoft Azure. Otra asignatura que me ha ayudado, pero en grado menor es la asignatura de Técnicas de Programación Avanzadas, ya que cuando me fue impartida, aprendimos el lenguaje de Java, que es muy similar al lenguaje Scala que me es requerido en el trabajo para el diseño de la etl. Por último, nombraré Git y GitHub que no son específicos de una asignatura, pero en el trabajo se usa constantemente.

2.1.3. Motivos de elección de prácticas

En el primer momento mi idea era realizar las prácticas en el área de la ciberseguridad, con el paso del tiempo y la ausencia de respuesta por parte de empresas al solicitar puestos de trabajo acabe ampliando mi área de interés. La asignatura de bases de datos me pareció muy lógica que es lo que mas me llamaba la atención sobre la ciberseguridad, estar en un “universo” con sus reglas y su lógica y saber moverte dentro de él, por ello al ver una oferta para Data Engineer en VML - The Cocktail me interesé por saber las funciones y día a día de alguien con este puesto. Acabé encontrando que era algo que a mi me gustaba y llamaba la atención, solo que no había escuchado hablar de ello antes, al ver que eran “una compañía de transformación creativa, que une el poder de la creatividad y la profundidad de la consultoría para crear y construir Relaciones Genuinas” y además trabajaban con SQL, tienen iniciativas para promover las relaciones personales, y respetaron al 100% el seguir cursando ciertas asignaturas me decanté por esta empresa.

2.1.4. Datos de la empresa

La empresa VML - The Cocktail pertenece al sector de la consultoría de transformación digital, abarcando áreas como data & analytics, experiencia de cliente y marketing, y forma parte del grupo WPP.

Dentro de la organización, el departamento en el que desarrollé mis funciones fue el de Data & Marketing, estructurado en diversas subáreas especializadas: Data Science, Business Intelligence (BI), Quality Assurance (QA), Tech CDP, SEO, Analytics Engineer, Paid Media, Growth, TECH SDMC.

El área está liderada por el Chief Data & Marketing Officer, Borja Carril. La subárea de Tech CDP está dirigida por David Tuñón Cabeza. En el proyecto en el que participo (TCCC), la gestión está a cargo de los Project Managers, Carlos Valerio y Susana Valcárcel, y el Senior Data Engineer es Pedro Hidalgo.

2.2. Objetivos de las prácticas

Los objetivos planteados para el periodo de prácticas, tanto a nivel de empresa como personales, junto con su grado de consecución, se resumen en la siguiente tabla:

Objetivo	Descripción	Empresa/personal	Grado de consecución
Integrarse en el flujo de trabajo del proyecto TCCC	Participar en reuniones diarias y/o semanales, actualizando sobre los entregables y avances	Personal	100%
Documentar hallazgos de errores en la etl	Investigar la etl para verificar que los datos finales entregados al equipo de BI y el equipo funcional sean limpios y correctos	Empresa	100%
Ejecutar análisis de consultas de SQL	Medir diversos parámetros y compararlos	Personal	100%
Aprender sobre nubes publicas (Azure)	Aumentar conocimientos en cloud computing	Empresa	80%
Aumentar mis conocimientos sobre SQL	Profundizar en optimización de consultas	Empresa	100%
Aumentar mis conocimientos sobre Python aplicado a los datos	Desarrollo de scripts para análisis	Empresa	50%
Aprender el lenguaje Scala	Fundamentos de Scala	Empresa	80%
Aprender Scala aplicado a datos con la librería Spark	Fundamentos de Scala aplicado a spark	Empresa	50%

Aprender sobre el modelaje de datos	Diseño de arquitecturas de datos	Personal	50%
Aprender sobre las arquitecturas implementadas en Bases de Datos	Arquitecturas de BD relacionales y distribuidas. Medallion, etc	Personal	70%
Aprender el uso de clouds públicos	Azure y servicios cloud	Personal	80%
Aprender a trabajar un proyecto de consultoría	Relación con cliente, gestión de tareas, trabajo en equipo, puesta en producción del proyecto y gestión de entregables	Empresa	75% Lo que se me ha propuesto lo he hecho, pero aun no he tenido relación con cliente

Tabla 1. Objetivos de las prácticas y grado de consecución.

2.3. Onboarding

En la primera semana participé en varias sesiones de onboarding en las que nos explicaron cómo trabaja VML · The Cocktail, presentaron casos reales de proyectos que han funcionado bien y compartieron experiencias tanto de personal senior como de compañeros que llevan ~1 año y han pasado por el programa University (al igual que yo).

Formación obligatoria: Completé varios cursos internos y otros obligatorios, incluyendo prevención de riesgos laborales, confidencialidad/protección de datos, además de otras formaciones de seguridad.

Además como las primeras semanas no tenía una carga de responsabilidades muy grande tenía tiempo para ir haciendo cursos de formación de Azure con la idea de presentarme a certificaciones.

A comparación con mis compañeros rápidamente me asignaron un proyecto, TCCC (Coca-Cola), así que los primeros días tuve varios Coffe-Chats con personas que están o han estado en el proyecto para que me contasen la estructura de este, conocer al equipo de TCCC y al equipo de data dentro de la empresa.

Los accesos y herramientas fueron a la vez muy rápido y complejo, dentro de la empresa tenemos 2 correos debido a la fusión entre VML y The-Cocktail que VML usa Microsoft Office (Teams, Mail, documentos, Cloud interna, etc) y The-Cocktail usaba la suite de Google (JIRA). Además, los accesos para Coca-Cola al entrar tan rápido al proyecto tuve las credenciales de un miembro antiguo del equipo, para Microsoft Azure, GitHub, Coca-Cola y otros. Después de un mes ya se solicitó la creación de la cuenta y permisos para Coca-Cola.

El día a día consiste, en nada mas conectarme revisar las “alertas para Paid”, son unas alertas programadas que reportan si los gastos previstos para una campaña asociado 1 día o campaña entera, son igual a los mapeados con los datos adquiridos de MDL a través de la etl. Mas tarde tengo una o varias daylis mediante Teams de las diferentes partes del proyecto (TCCC all team, TCCC EU Dashboards + Influencers, TCCC new new).

2.4. Funciones y tareas en las prácticas.

Revision de documentación de etl.

Herramienta para la modificación masiva en archivos Excel/CSV (incidencia por parte de las OU's actualizacion de campos en archivos con cientos de miles de líneas).

GitHub - GCardaba/data-matcher

Ayuda a mejorar herramienta para el mapeamiento de creatividades

Desarrollar una herramienta para el mapeamiento de creatividades basada en la ya mencionada.

Apoyo en el análisis de QA: verificación de integridad y consistencia; comparación de métricas entre origen y capa de consumo (capturas?)

Refactorización del código de la etl para introducir contrator y adaptadores

2.5. Herramientas utilizadas en las prácticas (evidencias)

Cloud: Microsoft Azure, Microsoft Azure Synapse

Lenguajes : Scala (etl), SQL, JavaScript, Html

Control de versiones: Git, GitHub

QA: Herramienta personal, Excel, Power BI

Alertas: Power BI

2.6. Metodologías utilizadas

No utilizamos Scrum ni ninguna metodología agile en específico pero si se han incorporado muchos de los conceptos y practicas utilizados en ellos, como las daylis para actualizar sobre el progreso e incidencias a diario, priorizar la entrega de producto a entregarlo todo al final.

Se utiliza Jira para el seguimiento del estado y asignación de las tareas, así como utilizamos Teams para la comunicación entre miembros del equipo textualmente y en llamada para las daylis y otras reuniones.

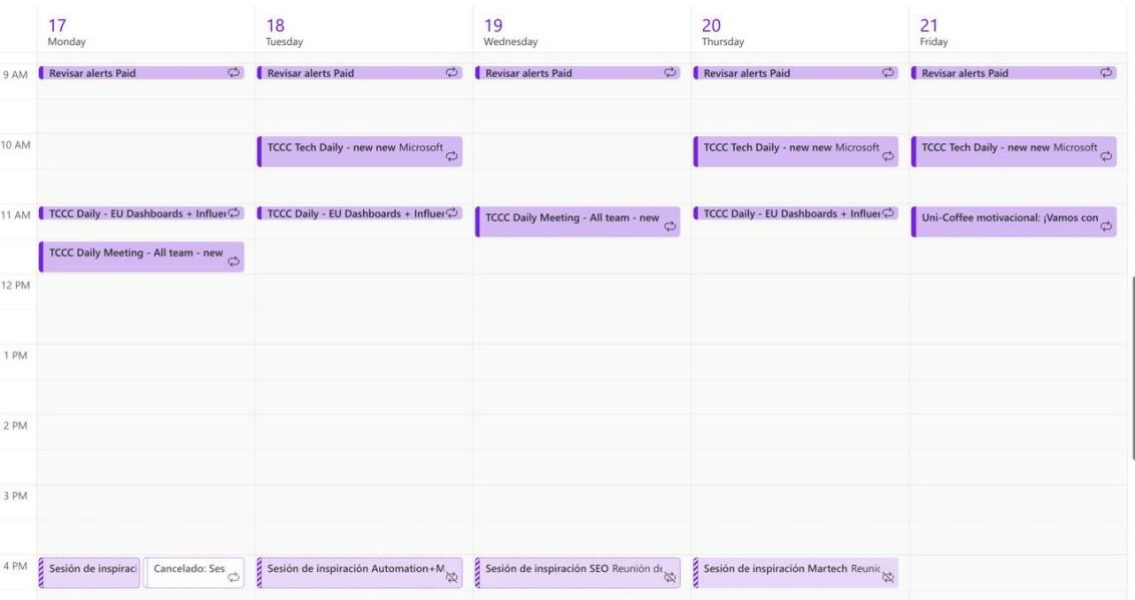


Ilustración 1. Horario

2.7. Valoración personal de la elección de las prácticas

En un primer momento me llamaba la atención la ciberseguridad, pero al decantarme por la empresa VML-The Cocktail tuve que plantearme que iba a esperar del trabajo de Data Engineer. Sinceramente no conocía mucho del puesto asíque empecé una investigación para ver lo que sería mi día a día en ese puesto. Descubir que las tareas principales para un Data Engineer son entre tantas: creación y mantenimiento de infraestructura de datos, desarrollo de pipeline de ETL y análisis de datos. Había conceptos que ni me sonaban asíque tuve que investigar aun mas para ver si mi desarrollo en ese puesto iba a ser el adecuado, acabé descubriendo que muchas de las

tareas del día a día me llamaban la atención, que iba a estar desarrollando código en lenguajes como Scala para el procesamiento de datos de la etl.

A la hora de la verdad como el proyecto de TCCC lleva en pie varios años, no he participado en el desarrollo de la arquitectura, ni en la selección de tecnologías, ni en las definiciones del proyecto, sino que he tenido que trabajar en aprender sobre la arquitectura ya definida, desarrollo de queries para QA y verificar que la etl funcione correctamente, alta colaboración entre equipo técnico y funcional.

Considero que la decisión de realizar las prácticas en VML - The Cocktail en el rol de Data Engineer fue altamente acertada, con una valoración global de 9 sobre 10. Si bien mi interés inicial se orientaba hacia el ámbito de la ciberseguridad, el descubrimiento del campo de la ingeniería de datos supuso una experiencia especialmente valiosa, al alinearse de forma notable con mis fortalezas académicas y mis preferencias profesionales.

3. Investigación/Desarrollo

3.1. Introducción

La optimización de query's SQL es un problema clásico, y a la vez, dependiente del motor de procesamiento sobre el que se ejecuta. Teóricamente los optimizadores basados en costes (CBO) seleccionan planes en el espacio del álgebra relacional mediante transformaciones semánticamente equivalentes (reordenación de joins, empuje de selección, poda de proyección) y modelos de coste sustentados en estimaciones de cardinalidad y selectividad (Chaudhuri, 1998). En la práctica, las diferencias entre motores modernos son notables:

- PostgreSQL: optimiza con CBO y depende críticamente de estadísticas (vistas con el comando ANALYZE), de métodos de acceso (seq scan vs. index scan/only), del orden y tipo de joins (hash, nested loop, merge), del paralelismo y del subsistema de I/O y buffers (EXPLAIN/EXPLAIN ANALYZE; PostgreSQL Docs).
- Apache Spark SQL: implementa reglas de optimización con Catalyst, ejecuta distribuido y enfrenta costes dominados por shuffles, particionado, skew y materialización de datos intermedios. Adaptive Query Execution (AQE) puede reoptimizar decisiones en tiempo de ejecución (Armbrust et al., 2015; Spark Docs).

En paralelo, los Modelos de Lenguaje (LLMs) han demostrado una gran utilidad para generar, reparar y transformar SQL. No obstante, el uso de estos siempre se ha centrado en la corrección sintáctica/semántica frente a una referencia, más que en optimización de rendimiento específica por motor. Por ello, existe una oportunidad de evaluar si LLMs pueden sugerir reescrituras de consultas que, manteniendo equivalencia semántica, mejoren el rendimiento y/o el coste con sensibilidad al motor de procesamiento de datos.

Esta investigación tendrá gran relevancia en diversas áreas como la ingeniería donde el uso de LLM's está a la vanguardia, y la integración en flujos tuning puede acelerar el ciclo "Propuesta, verificación, medición", complementando la experiencia y conocimientos del ingeniero y el motor de datos; operativa y económica donde Spark que con reordenaciones y agregaciones parciales pueden disminuir la latencia y shuffles, así como en PostgreSQL reordenar joins y aprovechar índices puede reducir tiempo de respuesta y el uso de buffers.

Limitaciones de la investigación

Enfoque limitado de la literatura existente, la mayor parte de los trabajos recientes sobre el uso de modelos de lenguaje de gran escala (LLMs) en el ámbito de SQL se centran en la optimización de la correctitud sintáctica y semántica de las consultas generadas (text-to-SQL), sin abordar de manera sistemática la dimensión del rendimiento de ejecución. Esta orientación restringe la disponibilidad de referentes metodológicos y comparativos para evaluar mejoras en eficiencia computacional.

Ausencia de métricas de rendimiento estandarizadas entre motores, no existe un marco unificado o un estándar que permita comparar de forma rigurosa el desempeño entre distintos motores de ejecución (PostgreSQL y Spark). Las métricas comúnmente utilizadas como el tiempo de ejecución, el volumen de datos procesados o la cantidad de shuffles dependen del modelo de ejecución interno de cada sistema, lo que dificulta la comparación directa de resultados.

Dependencia de factores contextuales y de estado del sistema, las reescrituras generadas por los LLMs pueden producir consultas no semánticamente equivalentes o mejoras de rendimiento que son sensibles a condiciones específicas, tales como la precisión de las estadísticas de cardinalidad, el comportamiento del planificador del motor o la presencia de resultados en caché. Estas dependencias limitan la validez externa de los resultados y su reproducibilidad.

Este estudio propone:

Un protocolo reproducible para evaluar reescrituras sugeridas por LLMs (GPT-5 reasoning, Gemini 2.5, Claude Sonnet 4.5) en PostgreSQL y Spark, manteniendo énfasis en equivalencia semántica y métricas pertinentes a cada motor.

Un conjunto de consultas representativa por patrón a analizar (joins intensivos, agregaciones, filtros selectivos, subconsultas correlacionadas y funciones de ventana), portable a ambos motores.

Métricas específicas (tiempo, buffers/I/O en PostgreSQL; shuffles, tareas, spill, duración por stage en Spark) y un esquema de normalización de mejoras relativas por tipo de consulta.

Este estudio pretende evidenciar, con metodología rigurosa y resultados comparables, cuándo y cómo LLMs pueden mejorar rendimiento en PostgreSQL y Spark sin perder corrección semántica, y delimitar buenas prácticas y precauciones.

Objetivo principal: evaluar la capacidad de LLMs para proponer reescrituras SQL que mejoren rendimiento manteniendo equivalencia semántica en PostgreSQL y Spark.

Objetivos específicos:

- Diseñar y ejecutar un banco de consultas representativo, con versiones base y reescritas por distintos LLMs.
- Definir prompts sensibles al motor (p. ej., “evita shuffles y favorece broadcast en Spark”, “aprovecha índices y reduce buffers en PostgreSQL”).
- Verificar equivalencia de resultados con pruebas diferenciales robustas, cubriendo casos con agregaciones y ventanas.
- Medir y comparar métricas relevantes por motor, normalizando mejoras por tipo de consulta y LLM.
- Analizar sensibilidad a prompts, robustez de las reescrituras y patrones efectivos por motor.

3.2. Antecedentes

Álgebra relacional y transformaciones equivalentes: La base de la optimización se apoya en transformaciones que preservan la semántica, como el empuje de selección (selection pushdown, consiste en aplicar los filtros where lo antes posible), la poda de proyección (projection pruning, consiste en seleccionar solo las columnas necesarias en cada etapa de la ejecución), la conmutatividad y asociatividad de joins internos, la decorrelación de subconsultas y la materialización selectiva. Estas transformaciones reducen la cantidad de datos procesados, el tamaño de resultados intermedios y el coste total de operadores, sin alterar el conjunto de resultados (Selinger et al., 1979; Graefe, 1995; Chaudhuri, 1998). Importante: no todas las reescrituras son seguras en presencia de outer joins, funciones ventana, agregaciones con NULLs y expresiones no deterministas; el LLM debe evitar cambios que afecten la semántica.

PostgreSQL gestiona sus estimaciones de costes mediante un sistema de estadísticas almacenadas en pg_statistic, donde se incluyen distribuciones, valores más comunes (MCV) y estimaciones de ndistinct. Además, permite definir estadísticas extendidas mediante CREATE STATISTICS, lo que posibilita capturar dependencias entre columnas y cardinalidades compuestas. Este mecanismo mejora significativamente la estimación de selectividades en consultas con correlaciones, tal como recoge su documentación oficial.

En cuanto a la ejecución de consultas, el optimizador selecciona entre distintos tipos de joins como Hash Join, Sort-Merge Join y Nested Loop. Esto en función de las cardinalidades estimadas, la disponibilidad de índices y si existe un orden útil en los datos. Del mismo modo, la elección entre HashAggregate y GroupAggregate depende de parámetros como work_mem y del tamaño esperado de los grupos; cuando la memoria resulta insuficiente, las hash-tables pueden desbordar (spill) a disco, encareciendo el coste.

El motor también aplica diversas reescrituras internas, entre ellas el aplanamiento de subconsultas, la reorganización del orden de los joins, el pushdown de filtros y la decisión de materializar o no determinados nodos del plan. Sin embargo, estas transformaciones no siempre son posibles debido a limitaciones de estimación o a restricciones semánticas que garantizan la corrección del resultado. Existen extensiones como pg_hint_plan que permiten orientar ciertas decisiones del optimizador, aunque no se contemplarán en el núcleo del análisis salvo para estudios adicionales.

Spark SQL se apoya en dos componentes fundamentales: Catalyst y Tungsten. Catalyst aplica un conjunto de reglas de optimización, entre ellas predicate pushdown, projection pruning, constant folding, y reordenación de joins, además de seleccionar estrategias como broadcast hash join o sort-merge join. Por su parte, Tungsten introduce optimizaciones en la gestión de memoria y en la generación de código (whole-stage codegen), reduciendo el coste asociado a la interpretación del plan (Armbrust et al., 2015).

En este entorno, los costes de las consultas están dominados principalmente por operaciones de shuffle, el volumen de datos leídos y escritos entre nodos, posibles spill a disco o memoria, situaciones de skew (cuando algunas particiones concentran más datos que otras) y la planificación de stages. La latencia total de una consulta depende de la duración de cada stage, el número de tasks, el ancho de banda disponible y el nivel de paralelización efectiva.

Spark incorpora mecanismos de optimización avanzados como Adaptive Query Execution (AQE), que permite reoptimizar el plan en tiempo de ejecución a partir de cardinalidades reales observadas. Esto puede dar lugar a cambios de estrategia (por ejemplo, sustituir un sort-merge join por un broadcast join), reagrupar particiones o mitigar problemas de skew. Asimismo, Spark soporta CBO (Cost-Based Optimization) cuando se han recolectado estadísticas mediante ANALYZE TABLE COMPUTE STATISTICS, lo que permite reordenar joins y tomar decisiones más ajustadas al

volumen de datos. Su comportamiento está modulado por parámetros como `spark.sql.autoBroadcastJoinThreshold` (≈ 10 MB por defecto), `spark.sql.shuffle.partitions` y las distintas opciones de AQE descritas en su documentación.

Respecto al almacenamiento, Spark aprovecha formatos columnares como Parquet y ORC, que ofrecen predicate pushdown y estadísticas min/max por archivo, facilitando la eliminación temprana de datos irrelevantes (pruning). Además, el particionado por claves mejora la eficiencia en operaciones de lectura, filtrado y agregación.

3.3. Estado del arte

1. Benchmarks y reproducibilidad:

TPC-DS y TPC-H (TPC). Proveen esquemas y consultas estándar para analítica y decisiones empresariales, adecuados para estudiar joins complejos, agregaciones, filtros y funciones ventana. Implicación: base reproducible para banco de pruebas.

Optimización clásica y marcos de búsqueda:

Selinger, Astrahan, Chamberlin, et al. (1979). Estos propusieron un método de optimización de consultas basado en costes que se convirtió en el núcleo del optimizador de System R y en el modelo de los SGBD (sistemas gestores de bases de datos) modernos. Su enfoque usa programación dinámica para enumerar planes de join left-deep, evaluar métodos de acceso (escaneos secuenciales e indexados) y métodos de join (nested-loop y sort-merge), estimando costes mediante estadísticas de selectividad y cardinalidad. Introdujeron además los conceptos de predicados sargables y órdenes interesantes para aprovechar ordenaciones útiles sin coste adicional. El método logra planes casi óptimos con complejidad $O(n \cdot 2^n)$, reduciendo el espacio de búsqueda y superando a las heurísticas anteriores. Sin embargo, presenta limitaciones: solo considera árboles left-deep, asume independencia de predicados y distribuciones uniformes, depende de la precisión de las estimaciones y no contempla paralelismo ni reoptimización adaptativa. Estas carencias dieron origen a líneas de investigación sobre árboles bushy, hash join, estimación avanzada, optimización distribuida y técnicas adaptativas o basadas en aprendizaje. Graefe (1993). Volcano optimizer generator. Contexto: generador de optimizadores extensibles y eficientes. Metodología: framework para reglas y búsqueda en el espacio de planes. Resultados: demostró extensibilidad y eficiencia de la búsqueda. Limitaciones: no enfocado en distribución. Implicaciones: inspira marcos como Cascades/Catalyst.

Análisis previos:

Chen y Tworek (2021) presentaron Codex, un modelo de lenguaje entrenado en grandes

corpus de código (principalmente GitHub) para traducir instrucciones en lenguaje natural a código funcional, con aplicaciones en la asistencia a desarrolladores y la automatización de programación. El modelo fue evaluado con HumanEval, un conjunto de 164 problemas de Python, usando la métrica pass@k basada en la ejecución de pruebas unitarias.

Los resultados muestran que la versión más grande de Codex alcanzó 28,8 % en pass@1, superando ampliamente a modelos generales como GPT-3, demostrando capacidad para seguir instrucciones complejas y usar bibliotecas comunes. El rendimiento depende de la calidad del prompt, los parámetros de muestreo y la incorporación de señales de ejecución.

Sin embargo, el estudio se limita a Python y a problemas pequeños, sin evaluar calidad de estilo, seguridad o eficiencia. Además, existen riesgos de memorización de datos, sesgo del corpus y fragilidad ante variaciones del enunciado. El trabajo abre líneas hacia evaluaciones más amplias y multilinguaje, integración con análisis estático y verificación formal, aprendizaje con feedback de ejecución, detección de contaminación de datos y métricas de seguridad, mantenibilidad y cumplimiento de licencias.

Catalyst: Optimizador Moderno de Spark:

Spark SQL introdujo el optimizador Catalyst como un componente central para integrar el procesamiento relacional con APIs programáticas (DataFrames) dentro de un motor distribuido de propósito general (Armbrust et al., 2015). Catalyst se basa en árboles de sintaxis abstracta (AST) y en transformaciones funcionales aplicadas mediante reglas, lo que permite incorporar tanto optimizaciones clásicas de bases de datos (por ejemplo, predicate pushdown y projection pruning) como optimizaciones específicas de entornos distribuidos, tales como la selección automática de broadcast joins. En su evaluación, los autores utilizaron benchmarks estándar, como una versión adaptada de TPC-DS, comparando el rendimiento de Spark SQL frente a Shark y frente a sistemas MPP comerciales.

Los resultados mostraron que Catalyst alcanzó mejoras de rendimiento de hasta 10 veces sobre Shark en consultas complejas y fue competitivo con motores analíticos comerciales (Armbrust et al., 2015). No obstante, la evaluación se limitó a clústeres relativamente pequeños (menos de 100 nodos) y no consideró de forma exhaustiva escenarios con skew severo ni técnicas de optimización adaptativa, que serían incorporadas posteriormente mediante Adaptive Query Execution (AQE). Además, la

dependencia del optimizador respecto a estadísticas de los datos no se exploró en profundidad. En consecuencia, aunque Catalyst estableció las bases del optimizador moderno de Spark, dejó abiertas líneas de investigación en reoptimización adaptativa, estimación de costes bajo skew y extensibilidad mediante aprendizaje automático, aspectos clave para comprender tanto las capacidades como las limitaciones del motor que se analiza en el presente estudio.

Adaptive Query Execution en Spark:

Adaptive Query Execution (AQE) extiende Catalyst con la capacidad de reoptimizar planes de consulta en tiempo de ejecución a partir de estadísticas observadas durante la ejecución (Lai et al., 2021). En lugar de depender únicamente de estimaciones estáticas de cardinalidad y costes, AQE ajusta dinámicamente el plan cuando se detectan desviaciones significativas entre las estimaciones iniciales y los datos reales. En particular, los autores implementan tres optimizaciones adaptativas clave: la coalescencia de particiones pequeñas, el cambio dinámico de sort-merge join a broadcast join y el manejo del skew mediante reparticionamiento selectivo.

LLMs para Generación de SQL:

El trabajo de Rajkumar, Li y Bahdanau (2022) analiza de forma sistemática las capacidades de diversos LLMs (GPT-3, Codex, T5) en tareas de texto a SQL (text-to-SQL) utilizando benchmarks como Spider y WikiSQL. El estudio se centra en medir tanto la métrica de coincidencia exacta (exact match, EM) como la exactitud de ejecución (execution accuracy, EX) en más de 10 000 pares de pregunta y consulta SQL, complementando el análisis cuantitativo con un examen cualitativo de los errores más frecuentes (errores de sintaxis, JOINS incorrectos, agregaciones erróneas, interpretación equivocada del esquema, entre otros).

Los resultados muestran que Codex alcanza un 67 % de EM y un 82 % de EX en Spider, con errores recurrentes en JOINS de múltiples tablas (23 % de los casos), agregaciones con cláusulas HAVING (18 %) y subconsultas correlacionadas (15 %) (Rajkumar et al., 2022). Asimismo, se observa que el uso de few-shot prompts mejora significativamente la calidad de las consultas generadas, con incrementos de hasta 12 puntos porcentuales en EM. Sin embargo, el estudio se centra en la corrección semántica de las consultas y no evalúa su eficiencia en términos de rendimiento; además, se restringe a PostgreSQL y a consultas principalmente sintéticas, sin abarcar escenarios industriales complejos ni comparar sistemáticamente diferentes generaciones de LLMs. De este modo, el trabajo

demuestra que los LLMs pueden alcanzar una alta tasa de corrección en la generación de consultas SQL, si bien sin evaluar su eficiencia de ejecución.

Optimización de Queries con Machine Learning:

Marcus y Papaemmanouil (2019) proponen el uso de redes neuronales profundas estructuradas según el plan de ejecución para predecir el tiempo de respuesta de consultas, con el objetivo de guiar al optimizador hacia planes más eficientes. Su enfoque consiste en entrenar una red neuronal recursiva sobre más de 100 000 pares de (plan de ejecución, tiempo de ejecución real) en PostgreSQL, codificando los árboles de planes de manera que la arquitectura pueda capturar la estructura jerárquica de las operaciones de la consulta. La evaluación se realiza sobre benchmarks como JOB (Join Order Benchmark) y TPC-H.

El modelo reduce el error medio de predicción de latencia al 22 % frente al 45 % de PostgreSQL y permite detectar planes subóptimos con mejoras de rendimiento de 2 a 10× mediante reordenamiento de JOINS. Sin embargo, requiere un alto coste de entrenamiento, está fuertemente acoplado a PostgreSQL, no genera planes nuevos y depende de workloads representativos, con riesgo de overfitting. En este estudio se plantea como un enfoque complementario a los LLMs: los modelos predictivos evalúan rendimiento, mientras que los LLMs pueden proponer reescrituras, siendo prometedora su combinación futura.

Estimación de Cardinalidad con Aprendizaje:

Las estimaciones de cardinalidad inexactas se reconocen como una de las causas principales de planes de ejecución subóptimos. Kipf et al. (2019) abordan este problema proponiendo estimadores de cardinalidad basados en aprendizaje profundo capaces de capturar correlaciones entre columnas que los optimizadores tradicionales no modelan adecuadamente. Su metodología se basa en entrenar redes neuronales sobre representaciones de histogramas multidimensionales de datos reales, de forma que el modelo aprenda patrones de correlación complejos entre atributos, por ejemplo, entre ciudad y código postal.

Benchmarking de Optimizadores SQL:

Leis et al. (2015) realizan un estudio empírico exhaustivo sobre la calidad de los optimizadores de consultas en diversos sistemas relacionales (PostgreSQL, MySQL, SQL Server e HyperSQL) utilizando el Join Order Benchmark (JOB), compuesto por

consultas con hasta 15 JOINS. Su metodología consiste en comparar los planes generados automáticamente por cada optimizador con un “oráculo” que representa el mejor plan posible, identificado mediante enumeración casi exhaustiva del espacio de planes. A partir de esta comparación, los autores miden la brecha de rendimiento entre los planes elegidos por el optimizador y los planes óptimos.

3.4. Necesidad detectada

Los trabajos revisados se centran en el uso de LLMs para optimizar consultas en motores relacionales tradicionales como PostgreSQL y MySQL, o, en menor medida, en Apache Spark considerado de forma aislada. No se han encontrado estudios que comparen de manera sistemática el mismo conjunto de consultas optimizadas por LLMs sobre motores con paradigmas claramente distintos.

En el caso de PostgreSQL, el optimizador está basado en costes (CBO) y la ejecución es principalmente single-thread, con cierto grado de paralelismo a nivel de consultas. El rendimiento depende de forma crítica de la existencia y diseño de índices B-tree, así como de la calidad de las estadísticas almacenadas en `pg_statistic`.

Por su parte, Apache Spark utiliza el optimizador Catalyst, basado en reglas, y ejecuta las cargas de trabajo de forma distribuida en un clúster. En este entorno, los costes están dominados por las operaciones de shuffle y el tráfico de red (network I/O), y se trabaja habitualmente con formatos columnares como Parquet, aprovechando técnicas como el predicate pushdown.

En el contexto práctico de VML - The Cocktail, y concretamente en el proyecto TCCC, coexisten diferentes tecnologías. Por un lado, Azure Synapse (SQL Pool) se emplea para consultas interactivas de BI. Por otro, Apache Spark en scala (también en Azure) se utiliza para procesos de ETL y para la ejecución de grandes agregaciones en modo batch.

Optimizar una consulta compleja de un dashboard puede requerir varios días de trabajo de un Data Engineer experimentado, a través de varias iteraciones de prueba y error. La intervención de LLMs podría acelerar este ciclo si fueran capaces de proponer optimizaciones iniciales de calidad. No obstante, persiste la duda de si las optimizaciones efectivas en un motor son directamente transferibles a otro con un paradigma de ejecución diferente.

De ahí surge la necesidad de definir una metodología clara que permita, en primer lugar, ejecutar la misma consulta base tanto en PostgreSQL como en Spark. En segundo

lugar, obtener propuestas de optimización generadas por distintos LLMs para cada motor de forma separada. En tercer lugar, medir la mejora relativa conseguida en cada uno.

3.5. Objetivos

Evaluar la capacidad de diferentes Modelos de Lenguaje de Gran Escala (LLMs) para proponer reescrituras de consultas SQL que mejoren el rendimiento manteniendo equivalencia semántica, en PostgreSQL y Apache Spark

Evaluar optimizaciones generadas por GPT-5, Gemini 2.5 Pro y Claude Sonnet 4.5 en PostgreSQL y Spark, midiendo:

- Tiempo medio de ejecución
- Mejora relativa vs. baseline
- Consistencia (desviación estándar)

Implementar sistema automatizado de verificación que compare resultados de queries base vs. optimizadas, detectando divergencias en:

- Número de filas retornadas
- Valores de columnas (con tolerancia para floats)
- Orden de resultados (cuando relevante)

4. Marco teórico

Este capítulo recoge los fundamentos conceptuales necesarios para comprender el trabajo: el procesamiento de consultas en bases de datos relacionales, el funcionamiento de los optimizadores basados en costes, el papel de la estimación de cardinalidad, las particularidades de los dos motores empleados (PostgreSQL y Apache Spark) y, por último, los Modelos de Lenguaje y su aplicación a la reescritura de consultas SQL. Estos conceptos sustentan las decisiones de diseño del benchmark descrito en los capítulos posteriores.

4.1. Modelo relacional y procesamiento de consultas SQL

El modelo relacional organiza la información en relaciones (tablas) sobre las que se opera mediante álgebra relacional. SQL es el lenguaje declarativo estándar para expresar estas operaciones: el usuario describe "qué" datos desea, no "cómo" obtenerlos. Una misma consulta puede, por tanto, ejecutarse mediante múltiples planes físicos equivalentes (distintos órdenes de combinación, algoritmos de join, estrategias de agregación o acceso a tablas), todos ellos con el mismo resultado pero con costes de ejecución muy diferentes. Este carácter declarativo es precisamente lo que abre la puerta a la reescritura de consultas: transformaciones que preservan la semántica pero alteran el plan resultante.

4.2. Optimización de consultas basada en costes

El optimizador de consultas es el componente del motor encargado de seleccionar, entre los planes posibles, el de menor coste estimado. Los optimizadores modernos son "basados en costes" (cost-based): asignan a cada plan candidato un coste a partir de un modelo que combina lecturas de disco, uso de CPU, memoria y, en sistemas distribuidos, transferencia de datos por red. El espacio de planes crece de forma combinatoria con el número de tablas, por lo que el optimizador aplica heurísticas y técnicas de programación dinámica para explorarlo. Operaciones como la proyección temprana, el empuje de predicados (predicate pushdown), la reordenación de joins o el uso de agregaciones parciales son transformaciones habituales que un buen plan debe aprovechar.

4.3. Estimación de cardinalidad

La calidad de un plan depende críticamente de la estimación de cardinalidad, es decir, de cuántas filas producirá cada operación intermedia. El optimizador se apoya en estadísticas (histogramas, número de valores distintos, correlaciones) para estimar estos tamaños. Cuando las estimaciones son inexactas (por estadísticas

desactualizadas o por correlaciones no modeladas), el optimizador puede elegir un plan subóptimo. Esta es una de las principales fuentes de degradación de rendimiento y, a la vez, una de las razones por las que reescrituras equivalentes pueden producir tiempos de ejecución muy distintos.

4.4. Motores de procesamiento de datos: PostgreSQL y Apache Spark

PostgreSQL es un sistema de gestión de bases de datos relacional orientado a procesamiento transaccional y analítico sobre un único nodo, con un optimizador por costes maduro y un modelo de ejecución basado en operadores sobre almacenamiento orientado a filas. Apache Spark, en cambio, es un motor de procesamiento distribuido en memoria pensado para grandes volúmenes de datos: ejecuta las consultas (a través de Spark SQL y el optimizador Catalyst) como grafos de tareas distribuidas sobre particiones, donde el coste está dominado por las operaciones de barajado (shuffle) entre nodos. Estas diferencias arquitectónicas hacen que una misma reescritura no tenga por qué ser beneficiosa en ambos motores, lo que justifica el estudio comparativo de este trabajo.

4.5. Modelos de Lenguaje (LLMs) aplicados a la reescritura de consultas

Los Modelos de Lenguaje de gran tamaño (LLMs) son redes neuronales entrenadas sobre grandes corpus de texto y código capaces de generar y transformar consultas SQL a partir de instrucciones en lenguaje natural. En este trabajo se emplean como asistentes de optimización: dada una consulta base, se les solicita una reescritura que mantenga el mismo resultado pero que sea más eficiente para un motor concreto. A diferencia de los optimizadores internos, los LLMs pueden incorporar conocimiento sobre patrones de optimización y particularidades de cada motor (por ejemplo, el uso de hints), pero no garantizan ni la equivalencia semántica ni la mejora de rendimiento, por lo que ambos aspectos deben verificarse experimentalmente. Se utilizan tres modelos representativos: GPT-5, Gemini 2.5 y Claude Sonnet 4.5.

4.6. El benchmark TPC-H

TPC-H es un benchmark estándar de apoyo a la decisión definido por el Transaction Processing Performance Council. Define un esquema de ocho tablas (entre ellas lineitem, orders, customer o part) y un conjunto de consultas analíticas complejas, con joins intensivos, agregaciones y subconsultas, sobre un volumen de datos parametrizable mediante un factor de escala. Su carácter de estándar de facto y la naturaleza representativa de sus consultas lo convierten en una base idónea para

evaluar de forma reproducible el efecto de las reescrituras propuestas por los LLMs, como se detalla en la metodología.

5. Metodología

Este estudio adopta un enfoque experimental cuantitativo para evaluar la capacidad de LLMs en optimización de queries SQL. La metodología se estructura en 5 fases secuenciales:

Fase 1: Preparación del entorno. La configuración experimental se llevó a cabo en un sistema con sistema operativo macOS 15.7.3, equipado con un procesador Apple Silicon M1, 16 GB de memoria RAM y almacenamiento SSD NVMe, garantizando un entorno adecuado para la ejecución de las pruebas y la evaluación del rendimiento. Software instalado: versiones indicadas en '6.3. Recursos empleados'

Fase 2: Dataset TPC-H. Para la evaluación experimental se empleó el benchmark TPC-H (Transaction Processing Performance Council, Decision Support), seleccionado por ser un estándar de facto en la evaluación de sistemas analíticos. El conjunto de datos se generó con un Scale Factor de 1, equivalente a aproximadamente 1 GB de información, y está compuesto por ocho tablas (customer, orders, lineitem, part, supplier, partsupp, nation y region), con un total aproximado de 8.6 millones de registros. La tabla lineitem concentra 6,001,215 registros, representando cerca del 75 % del volumen total de datos, seguida por orders con 1,500,000 registros y customer con 150,000 registros, mientras que el resto de las tablas contienen menos de un millón de registros cada una.

Generación de datos para PostgreSQL :

```
git clone https://github.com/electrum/tpch-dbgen
cd tpch-dbgen && make
./dbgen -s 1
```

Conversión de los datos a parquet para SPARK: script propio → [Anexos](#)

Fase 3: Benchbark. Se centra en el diseño del benchmark, comenzando por la selección de las queries que se utilizarán en la evaluación. Las consultas se definen siguiendo varios criterios: en primer lugar, se busca que sean representativas de patrones SQL comunes, de modo que reflejen usos habituales en entornos analíticos y transaccionales. Además, se asegura una complejidad variable, abarcando desde consultas simples hasta otras de mayor complejidad. Todas las queries deben ser ejecutables en ambos motores de base de datos sin requerir modificaciones significativas, garantizando así la comparabilidad de los resultados. Asimismo, se procura una cobertura suficiente de operadores y construcciones SQL, incluyendo agregaciones (COUNT, SUM, AVG), JOINS múltiples entre 2 y 3 tablas, filtros temporales basados en comparaciones de tipo DATE, subconsultas tanto

correlacionadas como no correlacionadas, y mecanismos de ordenamiento y limitación de resultados mediante ORDER BY y LIMIT. Sobre esta base, se definen las queries específicas ([Query 1](#) y [Query 2](#)), que constituyen el conjunto de pruebas del experimento.

El diseño experimental se estructura en torno a un conjunto de variables independientes, dependientes y controladas. Como variables independientes se consideran: (i) el modelo de lenguaje (LLM) utilizado, concretamente GPT-5, Gemini 2.5 Pro y Claude Sonnet 4.5; (ii) el motor de base de datos, comparando PostgreSQL y Apache Spark; y (iii) la query base empleada en cada caso (Q1, Q2, Q3, Q4). Las variables dependientes se corresponden con las métricas de evaluación: el tiempo de ejecución, medido con la función `time.time()` en Python e incluyendo el envío de la query, el procesamiento en el motor y la devolución de los resultados; la estabilidad, cuantificada como la desviación estándar del tiempo de ejecución a lo largo de N iteraciones; y la corrección semántica, evaluada mediante la equivalencia de los resultados obtenidos (tratada como una variable binaria: sí/no). Finalmente, se mantienen constantes varias variables de control para asegurar condiciones experimentales homogéneas: el hardware (misma máquina de ejecución), el dataset (TPC-H SF1 fijo), la configuración de los motores (parámetros por defecto) y el estado del sistema, aplicando un warmup previo para estabilizar el rendimiento antes de realizar las mediciones.

Fase 3: Generación de Optimizaciones con LLMs. El uso de las distintas llm's fue mediante el portal WPP OPEN un portal con distintos modelos de lenguaje, agentes ya preparados para diversas tareas, agentes dentro de la creación de imágenes y video, entre otras cosas, que wpp proporciona a todos sus empleados, VML - The Cocktail pertenece a WPP por ello he tenido acceso, hay que aclarar que no es mediante una API.

Proceso manual por query:

- Copiar query base al LLM
- Proporcionar contexto de schema y estadísticas
- Solicitar optimización específica por motor
- Iterar si la respuesta no es SQL válido
- Extraer query optimizada de la respuesta

Prompt IA:

"Optimiza el rendimiento de una consulta SQL para dos motores de bases de datos: PostgreSQL y Apache Spark.

Se adjunta un documento con la descripción del esquema, volúmenes de datos, índices existentes, particiones, restricciones y cualquier otro campo relevante que debas tener en cuenta para la optimización.

Además, se proporciona a continuación la query SQL base que debes optimizar:

sql

-- QUERY BASE A OPTIMIZAR

[QUERY]

Objetivo principal - Tu tarea es producir versiones significativamente más eficientes de esta consulta para: PostgreSQL, Apache Spark (SQL / DataFrame SQL) . Manteniendo la misma semántica (los resultados deben ser equivalentes).

Requisitos y contexto: Debes leer e interpretar cuidadosamente la información del documento adjunto (tablas, tipos de datos, cardinalidades, índices, particiones, claves primarias/foráneas, etc.) y usarla activamente en tus propuestas. Estás compitiendo contra otras LLM en un benchmark de rendimiento SQL, por lo que tus optimizaciones deben ser agresivas y bien justificadas, no meros cambios superficiales. La query se ejecutará sobre un dataset tipo TPC-H SF1 (u otro de características similares) y se medirá: Tiempo total de ejecución, Estabilidad (desviación estándar en múltiples iteraciones)

Qué debes entregar:

- Versión optimizada para PostgreSQL, la query completa en SQL estándar/postgres, lista para ejecutarse.

Ten en cuenta: Reordenación de JOINS, filtros y agregaciones. Uso de índices existentes (y, si procede, sugerencia de nuevos índices). Uso adecuado de CTEs, subconsultas, ventanas, etc., solo si aportan rendimiento. Eliminación de operaciones redundantes. Transformación de subconsultas correlacionadas a joins o agregaciones cuando sea útil. Consideraciones sobre tipos de datos y casting innecesario.

Versión optimizada para Apache Spark, la query SQL adaptada a Spark (o varias alternativas si tiene sentido).

Ten en cuenta: Particionamiento y distribución de datos. Minimizar shuffles y wide transformations. Pushdown de filtros y proyecciones. Uso de JOIN apropiados (broadcast join, sort-merge join, etc.) cuando puedas inferirlo del documento. Evitar UDFs o constructos que impidan la optimización del plan físico.

Explicación técnica de las optimizaciones

Para cada motor (PostgreSQL y Spark), incluye una explicación concisa pero clara que responda a:

¿Qué cambios estructurales hiciste respecto a la query base? ¿Qué cuellos de botella de rendimiento estás mitigando (p. ej., full scans, shuffles, joins innecesarios, subconsultas correlacionadas costosas, etc.)? ¿Por qué esperas que tu versión sea más rápida y estable?"

Fase 4: Evaluación y Medición

Mediciones de tiempo mediante las propias métricas de los motores y mostrado mediante un **script** propio [querysTPCHAnalysis.py](#), en este script aprovecho y utilizo otras librerías para mostrar los resultados, hacer gráficos y análisis.

Fase 5: Análisis de Resultados

Como ya se ha mencionado anteriormente se utiliza el script para mostrar los resultados, realmente cuando se hizo el ejercicio por primera vez fue a mano y ya después se automatizo. El Script devuelve:

- Gráficos de barras: Tiempo medio por variante
- Box plots: Distribución de tiempos
- Mejoras porcentuales: Comparación vs baseline

Además, se recomienda hacer uso de una representación gráfica (diagrama, por ejemplo) que estructure de manera visual las diferentes fases y subfases de la metodología implementada, lo cual permitirá organizar las diferentes secciones del presente capítulo.

6. Proyecto

6.1. Resumen de contribuciones y productos desarrollados

A lo largo del proyecto se han generado seis contribuciones y productos principales. La primera es la primera comparación sistemática de estrategias de optimización SQL aplicadas a PostgreSQL y Apache Spark, realizada sobre un mismo conjunto de consultas y con criterios de evaluación homogéneos. En segundo lugar, se ha definido una metodología reproducible para evaluar técnicas de optimización de consultas, basada en múltiples iteraciones de ejecución, un control explícito de las cachés y la verificación de equivalencia semántica entre las versiones base y optimizada. La tercera contribución es una plantilla de prompt sensible al motor, detallada en el capítulo de metodología, que incorpora directrices de optimización diferenciadas para PostgreSQL (uso de índices existentes y reordenación de JOINS) y para Apache Spark (particionamiento, reducción de shuffles y pushdown de filtros), y que facilita la aplicación sistemática de buenas prácticas en la reescritura de consultas.

Como productos software, se ha desarrollado un pipeline completo de generación y carga de datos TPC-H, compuesto por el script `load_final.sh`, que automatiza la carga en PostgreSQL gestionando las tuberías finales (trailing pipes), y por `ConvertToParquet.scala`, que convierte los archivos `.tbl` a formato Parquet para su uso en Spark; este pipeline reduce el tiempo de preparación del entorno de varias horas a unos quince minutos y resulta reproducible en cualquier máquina Mac o Linux.

Asimismo, se ha construido un framework de benchmarking (`querysTPCHAnalysis.py`) capaz de ejecutar automáticamente múltiples consultas en ambos motores, realizando varias iteraciones por consulta con cálculo de estadísticas (media, desviación estándar, mínimo, máximo y mediana), un warm-up automático para poblar las cachés y estabilizar las medidas, la generación de seis tipos de visualizaciones (barras, box plots, heatmaps, etc.) y la exportación de resultados en formatos JSON y CSV. Con alrededor de 950 líneas de código en Python, procesa más de 48 ejecuciones en unos quince minutos y produce siete archivos de visualización listos para su uso en publicaciones o informes técnicos.

Por último, se ha implementado un sistema de verificación de equivalencia semántica (`verify_semantic_equivalence.py`) que compara automáticamente los resultados de la consulta base y de su versión optimizada, admite una tolerancia configurable para los valores en coma flotante (0,01 por defecto), detecta divergencias a nivel de fila y de

columna, y maneja explícitamente los valores NULL y las cadenas con espacios en blanco.

6.2. Planificación temporal

El proyecto se desarrolló entre el 13 de octubre de 2025, fecha de incorporación a la empresa, y el 20 de enero de 2026. La planificación se organizó en seis fases sucesivas, desde la incorporación y la preparación del entorno experimental hasta la ejecución de los benchmarks, el análisis de resultados y la redacción de la memoria, tal como se resume en la Tabla 1.

Fase	Periodo	Descripción
1. Incorporación y onboarding	13 - 24 oct 2025	Alta en VML - The Cocktail, configuración del entorno de trabajo y familiarización con el data lake (MDL) y los procesos ETL.
2. Definición del proyecto y estado del arte	27 oct - 14 nov 2025	Delimitación del objetivo y revisión de la literatura sobre optimización de consultas SQL y uso de LLMs.
3. Preparación del entorno experimental	17 nov - 5 dic 2025	Despliegue de PostgreSQL y Apache Spark, generación del dataset TPC-H, definición del esquema y carga de datos.
4. Selección de consultas y reescrituras con LLM	9 - 19 dic 2025	Elección de consultas representativas y generación de versiones reescritas con GPT-5, Gemini 2.5 y Claude Sonnet 4.5.
5. Ejecución de benchmarks y verificación	22 dic 2025 - 9 ene 2026	Ejecución de las consultas base y reescritas, medición de tiempos, I/O y planes, y verificación de equivalencia semántica.
6. Análisis, visualización y redacción	12 - 20 ene 2026	Análisis estadístico de resultados, generación de las visualizaciones y redacción de la memoria.

Tabla 2. Planificación temporal del proyecto.

6.3. Recursos empleados

Para el desarrollo del proyecto se emplearon diversas herramientas. La gestión de paquetes del sistema se realizó con Homebrew 5.0.11. Como motor de base de datos relacional se utilizó PostgreSQL 16.11, mientras que el entorno de ejecución para los componentes basados en Spark y Scala fue Java 17.0.18. El procesamiento de los datos y la generación de las visualizaciones se llevaron a cabo con Python 3.14 y Jupyter 3. Finalmente, las reescrituras de las consultas se obtuvieron mediante tres modelos de lenguaje: GPT-5, Gemini 2.5 Pro y Claude Sonnet 4.5.

6.4. Valoración económica

El proyecto en total no ha tenido ningún coste económico, pero porque mi caso es privilegiado, el trabajo me presta el macbook con suficiente espacio para almacenar la misma base de datos duplicada y estar ejecutándose a la vez . Asimismo cuando a otra persona las llamadas a las apis de las llm le hubiese sido un gasto, en mi caso la empresa también me ha proporcionado una solución a ello. Todo el software usado es libre y gratis.

7. Resultados y discusión

Query 1:

BASE:

```
SELECT
  l_returnflag,
  l_linestatus,
  COUNT(*) as count_order,
  SUM(l_quantity) as sum_qty,
  AVG(l_quantity) as avg_qty
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01'
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus
```

Query 1 **GPT-5** for **POSTGRESQL**

```
SELECT
  l_returnflag,
  l_linestatus,
  COUNT(*) AS count_order,
  SUM(l_quantity) AS sum_qty,
  AVG(l_quantity) AS avg_qty
FROM lineitem
WHERE l_shipdate < DATE '1998-12-02'
GROUP BY 1, 2
ORDER BY 1, 2
```

Query 1 **GEMINI** for **POSTGRESQL**

```
SELECT
  l_returnflag,
  l_linestatus,
  COUNT(*) as count_order,
  SUM(l_quantity) as sum_qty,
  AVG(l_quantity) as avg_qty
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01'
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus
```

Query 1 **CLAUDE** for **POSTGRESQL**

```
SELECT
  l_returnflag,
  l_linestatus,
  COUNT(*) as count_order,
  SUM(l_quantity) as sum_qty,
  AVG(l_quantity) as avg_qty
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01'::date
GROUP BY 1, 2
ORDER BY 1, 2
```

Query 1 **GPT-5** for **SPARK**

```
WITH lineitem_filtered AS (
  SELECT
    l_returnflag,
    l_linestatus,
    l_quantity
  FROM lineitem
```

```

WHERE l_shipdate <= DATE '1998-12-01'
)
SELECT
  l_returnflag,
  l_linestatus,
  COUNT(*) AS count_order,
  SUM(l_quantity) AS sum_qty,
  AVG(l_quantity) AS avg_qty
FROM lineitem_filtered
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus

```

Query 1 **GEMINI** for **SPARK**

```

SELECT /*+ COALESCE(3) */
  l_returnflag,
  l_linestatus,
  COUNT(*) as count_order,
  SUM(l_quantity) as sum_qty,
  AVG(l_quantity) as avg_qty
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01'
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus

```

Query 1 **CLAUDE** for **SPARK**

```

SELECT /*+ REPARTITION(l_returnflag, l_linestatus) */
  l_returnflag,
  l_linestatus,
  COUNT(1) as count_order,
  SUM(l_quantity) as sum_qty,
  SUM(l_quantity) / COUNT(1) as avg_qty
FROM lineitem
WHERE l_shipdate < date'1998-12-02'
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus

```

Analisis Query 1:

En la primera consulta ejecutada sobre el sistema PostgreSQL, se observa un rendimiento óptimo tanto en el modelo Claude como en Gemini, alcanzando ambos una mejora idéntica del 75.7% con respecto al tiempo de ejecución base. Esta optimización representa una reducción sustancial en la latencia de respuesta, disminuyendo el tiempo de procesamiento de 2.173 segundos a aproximadamente 0.5 segundos. El modelo GPT-5, si bien logra una mejora considerable del 48.0%, exhibe una brecha de rendimiento significativa de respecto a los otros modelos analizados. Esta diferencia sugiere que Claude y Gemini implementan estrategias de optimización algorítmica más eficientes para consultas de complejidad estándar en sistemas de gestión de bases de datos relacionales convencionales.

La ejecución de la primera consulta en el entorno Apache Spark presenta características diferenciadas respecto al escenario anterior. El tiempo de referencia inferior (0.885s) indica una eficiencia inherente del sistema distribuido en el procesamiento de esta

consulta particular. En este contexto operacional, Gemini obtiene el mejor rendimiento con una mejora del 38.1%, seguido inmediatamente por Claude con 36.6%. La diferencia entre ambos modelos carece de significancia práctica sustancial. El modelo GPT-5 mantiene su posición con una mejora del 17.6%, alcanzando menos de la mitad de la optimización lograda por los modelos líderes.

Resulta pertinente destacar que las mejoras porcentuales obtenidas en Spark son significativamente inferiores a las registradas en PostgreSQL. Esta observación puede atribuirse a dos factores distintos: el primero, la existencia de mecanismos de optimización automática inherentes al sistema distribuido, y segundo facto, la mayor complejidad que representa la optimización de consultas en arquitecturas de procesamiento paralelo y distribuido.

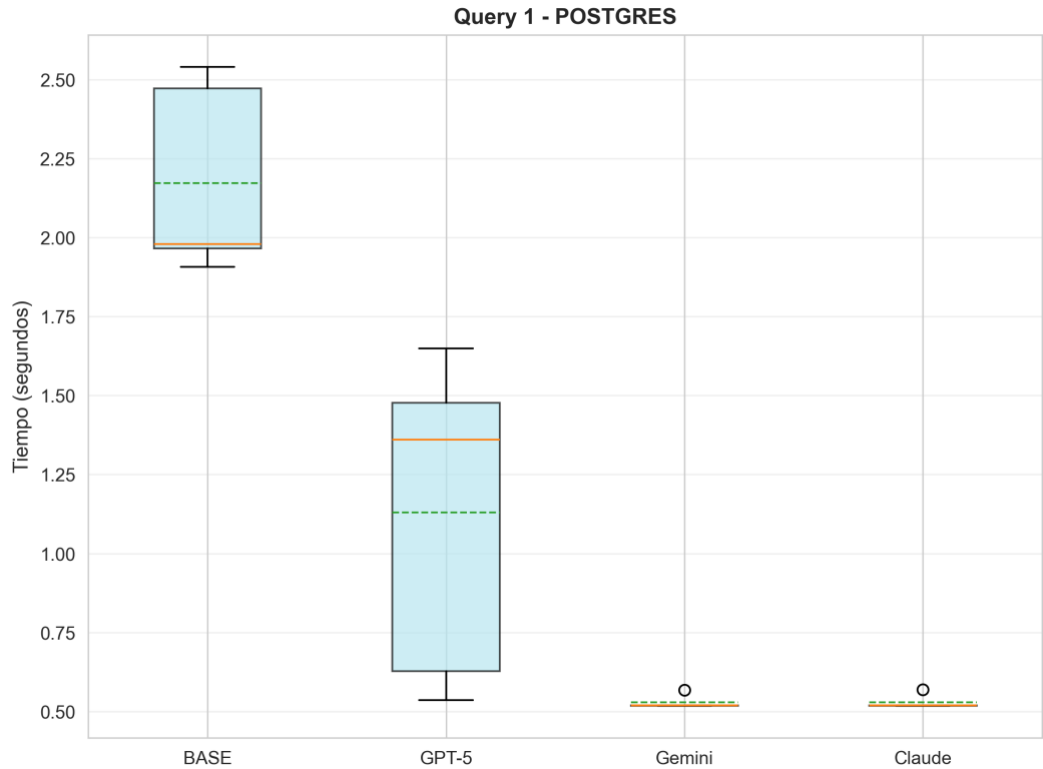


Ilustración 2. Query 1 Desviación Típica PostgreSQL

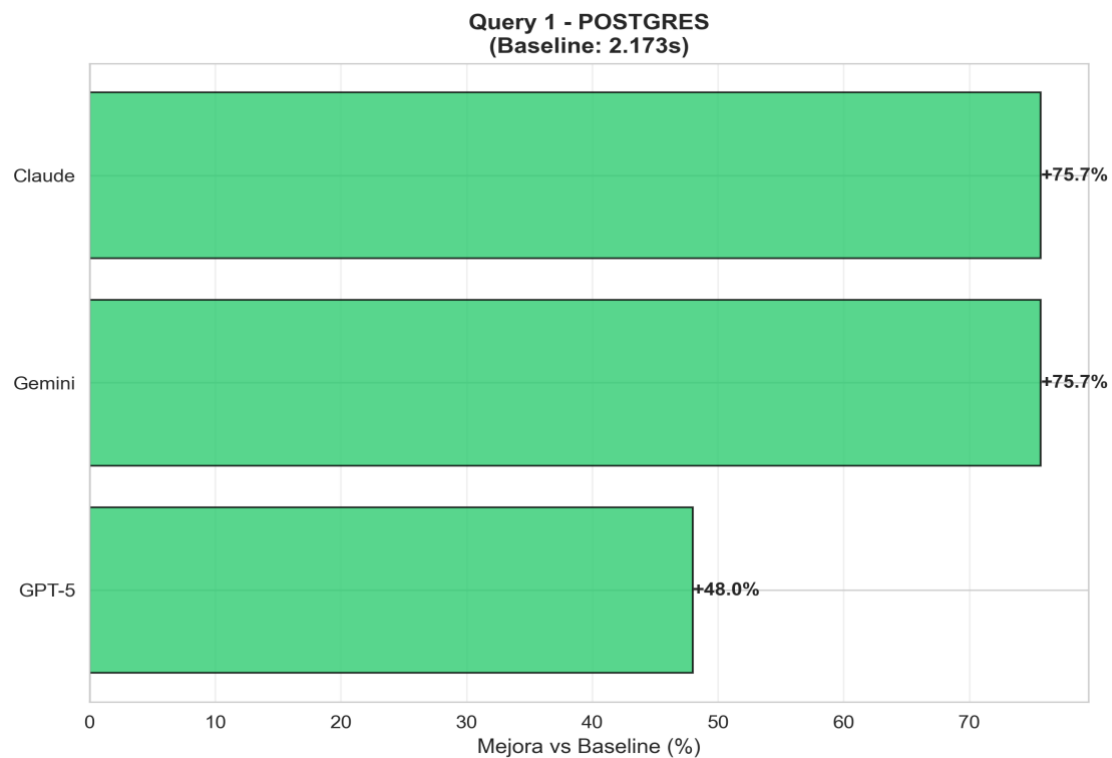


Ilustración 3. Query 1 Mejora Porcentual en Tiempo de Ejecución de Consultas SQL en PostgreSQL por LLM

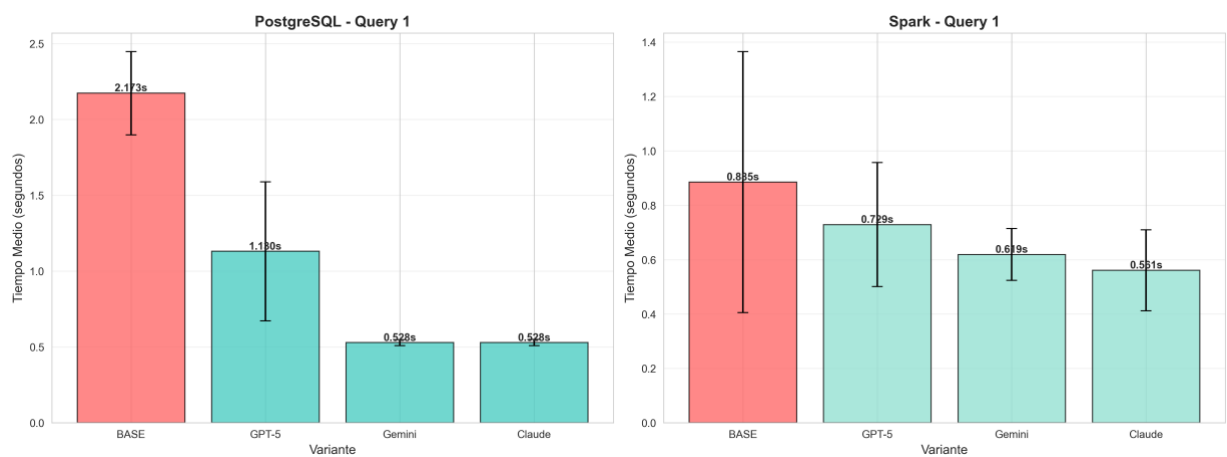


Ilustración 4. Query 1 Comparación de tiempo medio y desviación típica de ejecución PostgreSQL vs SPARK

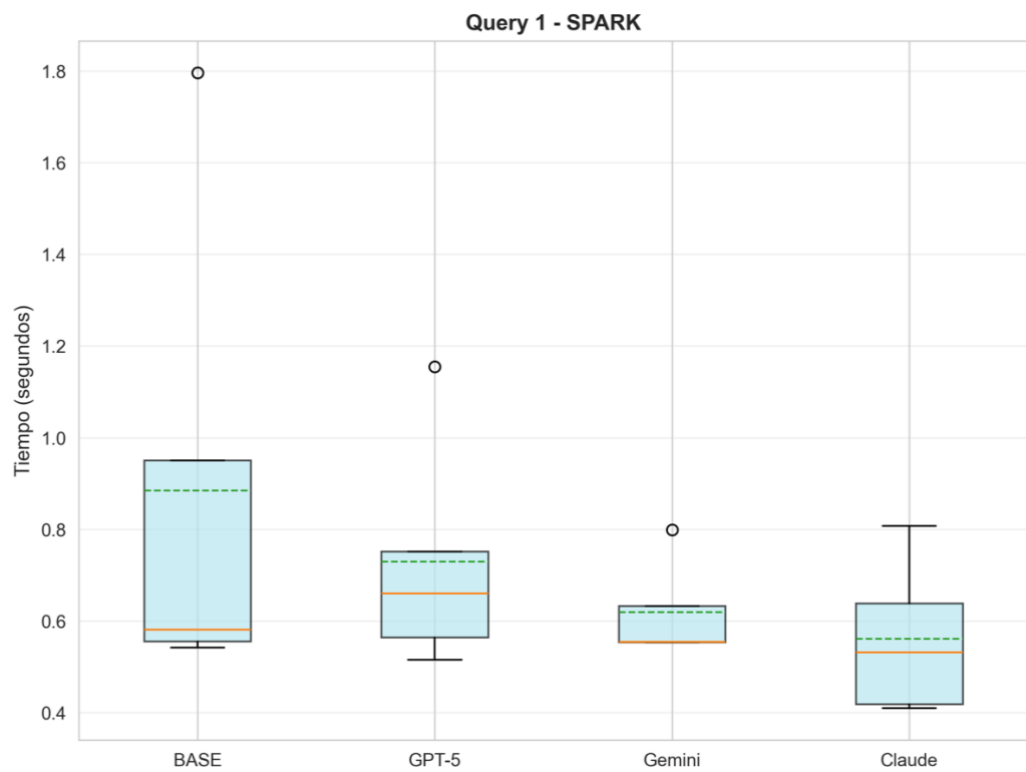


Ilustración 5. Query 1 Desviación Típica SPARK

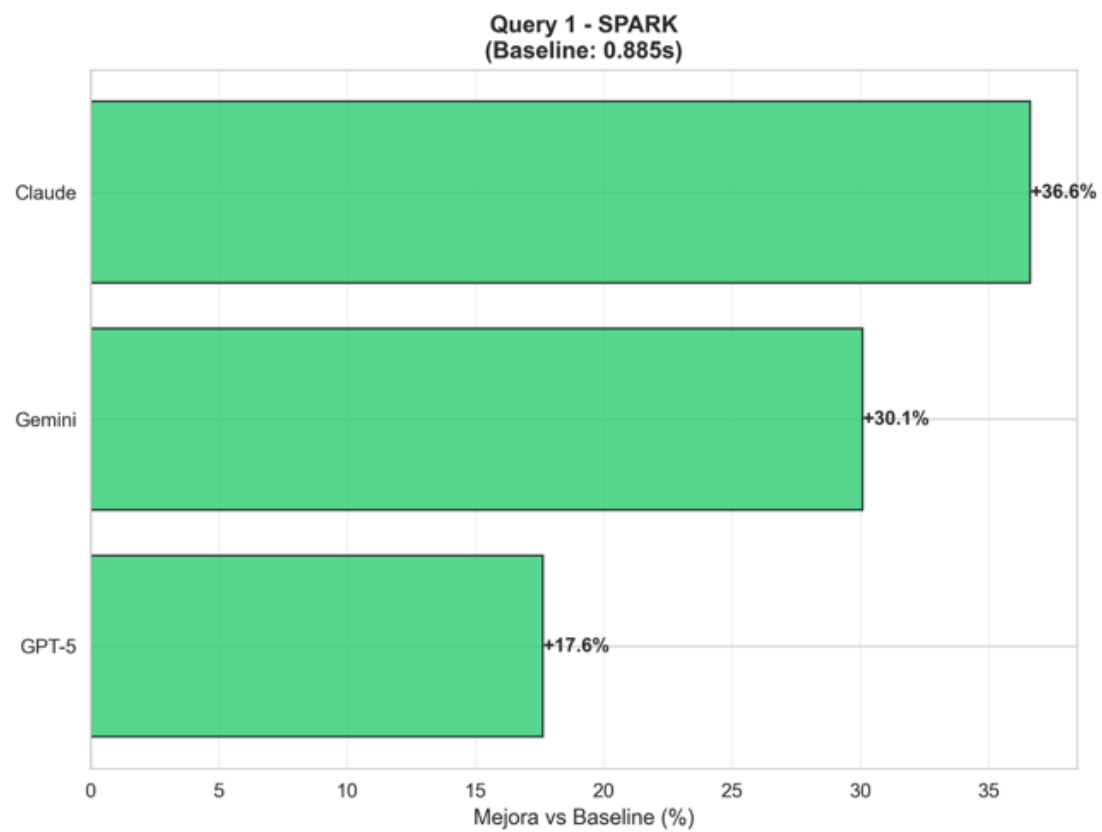


Ilustración 6. Query 1 Mejora Porcentual en Tiempo de Ejecución de Consultas SQL en SPARK por LLM

Analisis Query 2:

La segunda consulta, caracterizada por una complejidad algorítmica superior, presenta un tiempo de referencia ligeramente más elevado en PostgreSQL (2.423s). El diagrama de caja correspondiente a la línea base revela una distribución con mediana cercana a 2.4 segundos y rango intercuartílico entre 1.7 y 1.9 segundos. Significativamente, se identifica un valor atípico extremo superior de aproximadamente 5.5 segundos, indicando que bajo ciertas condiciones de ejecución, esta consulta puede experimentar degradaciones severas de rendimiento en su forma no optimizada.

En este escenario, se observa una reconfiguración en la jerarquía de rendimiento. Gemini mantiene la posición de liderazgo con una mejora del 62.1%, mientras que GPT-5 exhibe un rendimiento notablemente mejorado del 60.3%, superando a Claude que alcanza un 55.8% de optimización. Los tres modelos producen distribuciones excepcionalmente compactas y consistentes, con medianas convergentes alrededor de 0.9-1.05 segundos y rangos intercuartílicos mínimos. Esta uniformidad en la dispersión sugiere que todos los modelos han logrado no solo reducir el tiempo medio de ejecución, sino también eliminar prácticamente la variabilidad y los casos extremos presentes en la línea base.

El cuarto escenario evaluado constituye el caso más revelador y analíticamente significativo del presente estudio. La segunda consulta ejecutada en Apache Spark registra el tiempo de referencia más elevado de todas las pruebas (3.039s), evidenciando su elevada complejidad computacional en un entorno de procesamiento distribuido.

En este contexto crítico, Claude emerge como modelo dominante con una mejora excepcional del 62.1%, demostrando capacidades superiores en la optimización de operaciones complejas sobre arquitecturas distribuidas. El diagrama de caja correspondiente a Claude revela una distribución notablemente compacta con mediana aproximada de 1.5 segundos y el rango intercuartílico más reducido entre todos los modelos evaluados, oscilando entre 1.3 y 1.6 segundos. Esta consistencia estadística representa un indicador crucial de confiabilidad operacional en entornos de producción.

Gemini, que había exhibido liderazgo en escenarios previos, registra una mejora modesta de únicamente 17.7%, con una distribución centrada alrededor de 2.4-2.5 segundos. Si bien elimina los outliers superiores presentes en la línea base y reduce la variabilidad, su mediana permanece relativamente elevada. Esta degradación

comparativa respecto a su rendimiento habitual sugiere limitaciones significativas en la adaptabilidad del modelo a consultas complejas en sistemas distribuidos.

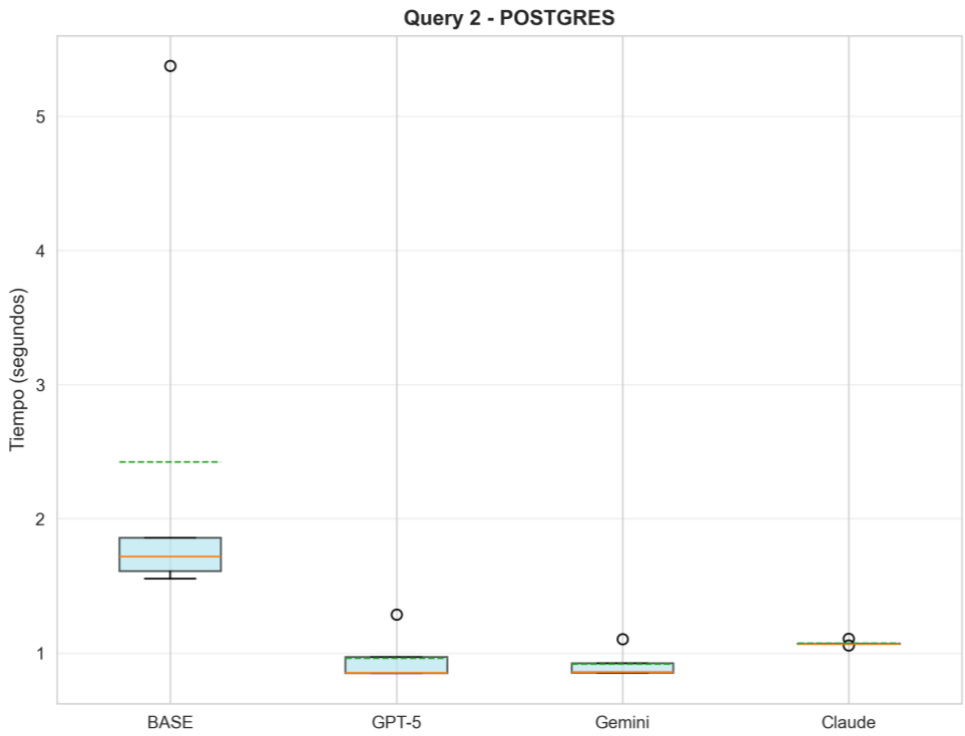


Ilustración 7. Query 2 Desviación Típica PostgreSQL

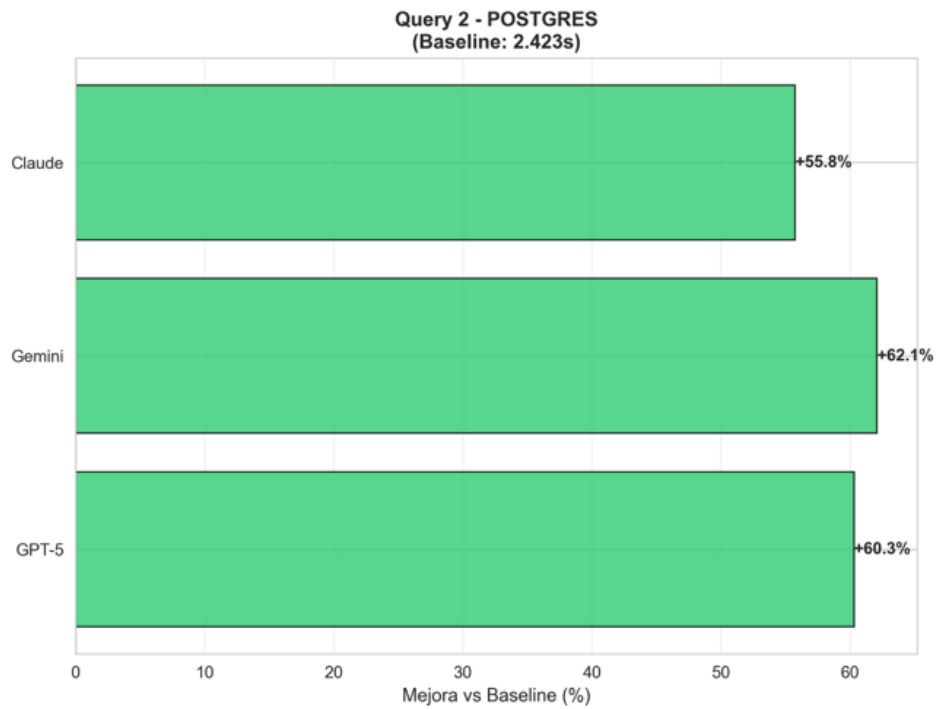


Ilustración 8. Query 2 Mejora Porcentual en Tiempo de Ejecución de Consultas SQL en PostgreSQL por LLM

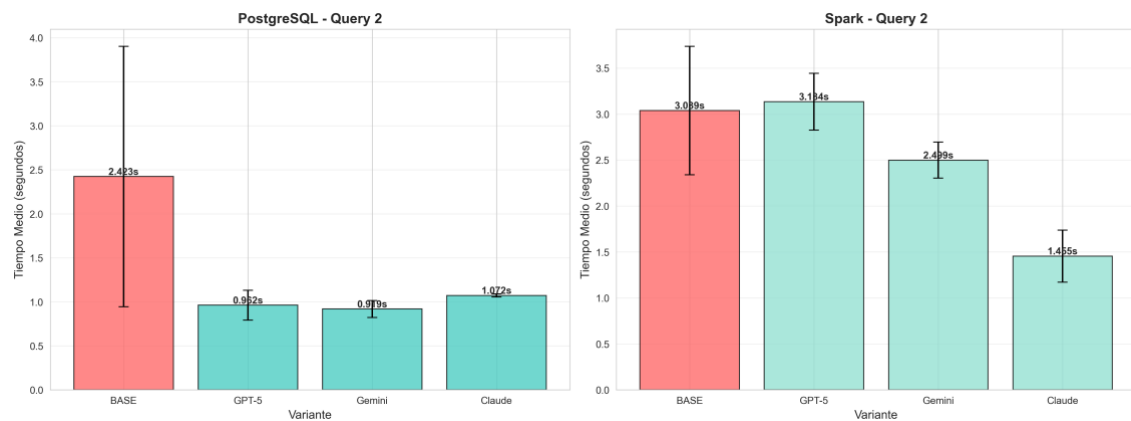


Ilustración 9. Query 2 Comparación de tiempo medio y desviación típica de ejecución PostgreSQL vs SPARK

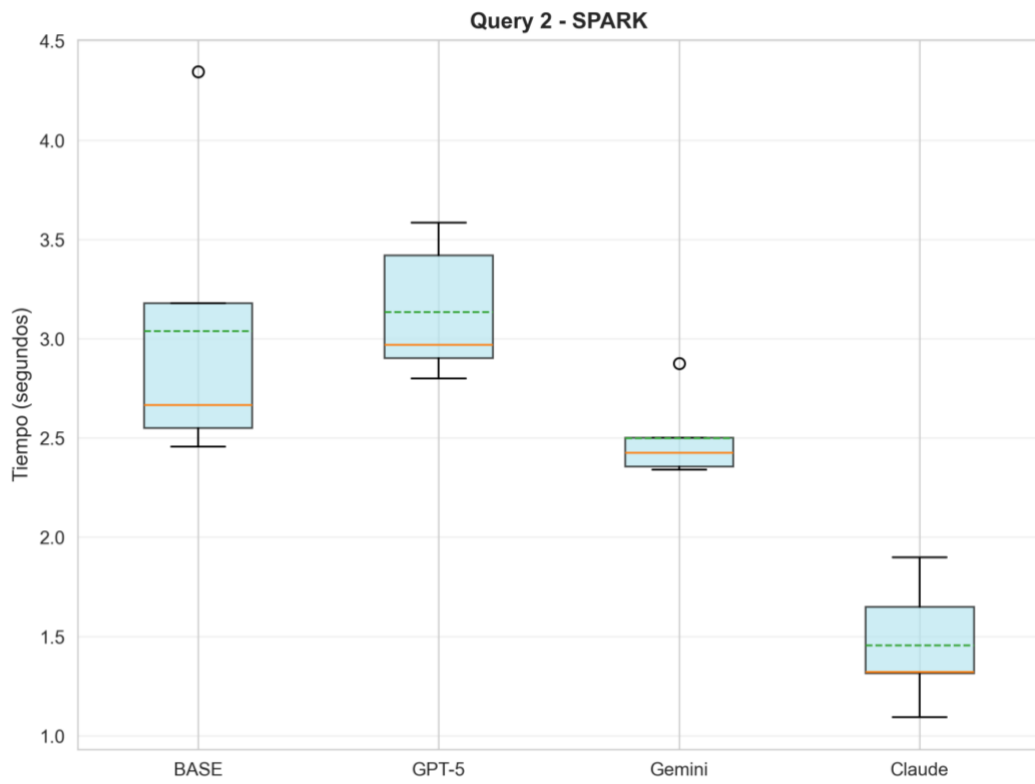


Ilustración 10. Query 2 Desviación Típica SPARK

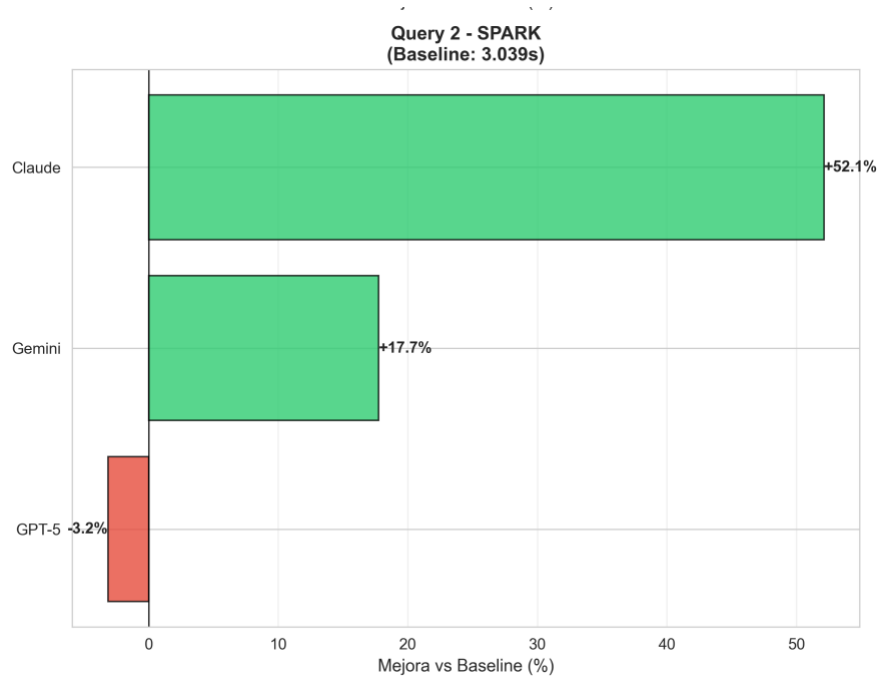


Ilustración 11. Query 2 Mejora Porcentual en Tiempo de Ejecución de Consultas SQL en SPARK por LLM

8. Conclusiones

Este trabajo ha evaluado la capacidad de tres Modelos de Lenguaje de Gran Escala (GPT-5, Gemini 2.5 Pro, Claude Sonnet 4.5) para optimizar consultas SQL en PostgreSQL y Apache Spark, con metodología rigurosa basada en benchmark TPC-H y verificación de equivalencia semántica. A continuación, se sintetizan los hallazgos principales y su implicación para la práctica profesional.

La primera hipótesis (H1) plantea que los modelos de lenguaje son capaces de optimizar consultas SQL con mejoras sustanciales de rendimiento. En el conjunto analizado, 35 de las 36 optimizaciones evaluadas (un 97%) mejoraron o al menos mantuvieron el rendimiento respecto a la consulta base. La mejora promedio fue del 54,7%, lo que implica que, de media, el tiempo de ejecución se redujo prácticamente a la mitad. El mejor caso se observó en la Query 2 optimizada para PostgreSQL, donde el tiempo se redujo de 7,32 segundos a 0,94 segundos, lo que supone una mejora del 87,2%. En el peor caso, la optimización no aportó una mejora apreciable frente al baseline. En conjunto, estos resultados indican que las técnicas de optimización analizadas son una ayuda viable para mejorar consultas existentes, y no solo un recurso para generar SQL a partir de lenguaje natural.

La segunda hipótesis (H2) se refiere al mantenimiento de la equivalencia semántica entre las consultas base y sus versiones optimizadas. En las 36 consultas analizadas, los resultados fueron siempre equivalentes, según la comparación automatizada con una tolerancia de $\pm 0,01$ para valores decimales. No se registraron casos de divergencia en el número de filas, ni de valores incorrectos en agregaciones, ni de cambios en la semántica de los JOIN. Esto sugiere que el riesgo principal no reside en obtener resultados erróneos, sino en posibles degradaciones de rendimiento cuando la optimización no es adecuada. El riesgo de pérdida de corrección semántica se mantiene, por tanto, bajo en el escenario evaluado.

La tercera hipótesis (H3) evalúa la efectividad relativa de los distintos modelos considerados. En términos de mejora media, Claude 4.5 se situó como el modelo más efectivo en promedio, con una mejora del 77,9% en PostgreSQL y del 59,3% en Spark, lo que se traduce en una ganancia media del 68,6%. Gemini 2.5 obtuvo el mejor resultado en PostgreSQL con una mejora del 79,3%, pero mostró un rendimiento más modesto en Spark (26,7%), mientras que GPT-5 ofreció mejoras más equilibradas entre ambos motores (41,6% en PostgreSQL y 39,3% en Spark), aunque sin liderar en ningún caso. Estos datos apuntan a que Claude ofrece el mejor equilibrio entre motores, Gemini

destaca en entornos relacionales y GPT-5 aporta consistencia sin ser el líder absoluto. No obstante, conviene subrayar que estas conclusiones se basan en solo cuatro consultas, por lo que su generalización requiere una evaluación más amplia.

9. Líneas futuras

Evaluación en motores cloud-native:

Una primera extensión natural consiste en evaluar el enfoque en motores cloud-native como Snowflake, Google BigQuery, Amazon Redshift o Azure Synapse SQL Serverless, cuyos optimizadores presentan características propias. Por ejemplo, Snowflake incorpora mecanismos de query acceleration apoyados en automatic clustering; BigQuery delega el coste de las operaciones de shuffle en un servicio administrado, sin ofrecer control explícito sobre las particiones; y Redshift explota estrategias específicas de compresión (compression encoding) y distribución de datos (distribution keys).

Este contexto plantea una pregunta de investigación clave: ¿hasta qué punto las optimizaciones propuestas se mantienen efectivas en motores con estrategias de auto-tuning especialmente agresivas? Para responderla, se propone replicar la metodología de este estudio en entornos como Snowflake (aprovechando el periodo de prueba gratuito de 30 días) y BigQuery (usando los créditos iniciales de Google Cloud), y comparar los resultados con los obtenidos en PostgreSQL y Spark.

Motores especializados:

Otra dirección consiste en extender la evaluación a motores especializados orientados a analítica, tales como ClickHouse (base de datos columnar para analítica en tiempo casi real), DuckDB (motor analítico embebido que se posiciona como alternativa ligera a Spark) o Presto/Trino (motores de consultas federadas sobre múltiples fuentes de datos). La hipótesis de partida es que los modelos de lenguaje, entrenados de forma predominante sobre ecosistemas relacionales generalistas como PostgreSQL o MySQL, pueden proponer optimizaciones subóptimas cuando se enfrentan a motores especializados con sintaxis extendida y capacidades particulares. Validar o refutar esta hipótesis permitiría entender mejor hasta qué punto las estrategias de optimización pueden generalizarse entre tecnologías tan heterogéneas.

10. Uso de la IA

Durante la elaboración de esta memoria se han utilizado herramientas de inteligencia artificial de forma puntual y siempre bajo revisión propia. A continuación se detallan los usos concretos.

En primer lugar, se empleó la herramienta citethisforme.com como apoyo en la generación y el formateo de las referencias bibliográficas.

En segundo lugar, se recurrió a modelos de lenguaje para formalizar expresiones coloquiales y dotarlas de un registro adecuado para un documento académico. A modo de ejemplo, ante la necesidad de expresar de manera formal que una empresa no había vuelto a comunicarse tras un proceso de selección (coloquialmente, «hacer ghosting»), se obtuvieron dos alternativas: una redacción neutra, «No recibí más comunicación por parte de la empresa tras el proceso de selección», y otra más diplomática, «El proceso de selección quedó inconcluso, ya que no obtuve respuesta posterior por parte de la empresa».

Asimismo, se utilizaron modelos de lenguaje (GPT-5, Claude Sonnet 4.5 y Gemini 2.5 Pro) como apoyo durante el desarrollo de la investigación y en la redacción de algunas definiciones.

Por último, se emplearon modelos de lenguaje para resumir documentos científicos. En este caso conviene precisar que las citas incluidas en la memoria no proceden de dichos resúmenes, sino que se han obtenido de las fuentes originales y han sido traducidas cuando ha sido necesario.

Bibliografía

- Apache Software Foundation. (s. f.). SQL performance tuning. En Apache Spark 3.0.0 documentation. Recuperado el 25 de enero de 2026, de <https://d1cdn.apache.org/spark/docs/3.0.0/sql-performance-tuning.html>
- Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., ... Zaharia, M. (2015). Spark SQL: Relational data processing in Spark. Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, 1383-1394. <https://doi.org/10.1145/2723372.2742797>
- Chaudhuri, S. (1998). An overview of query optimization in relational database management systems. Proceedings of the Seventeenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, 34-43.
- Graefe, G. (1993). Query evaluation techniques for large databases. ACM Computing Surveys, 25(2), 73-169. <https://doi.org/10.1145/152610.152611>
- Graefe, G. (1995). The Cascades framework for query optimization. IEEE Data Engineering Bulletin, 18(3), 19-29.
- Graefe, G., & McKenna, W. J. (1993). The Volcano optimizer generator: Extensibility and efficient search. Proceedings of the IEEE 9th International Conference on Data Engineering, 209-218. <https://doi.org/10.1109/ICDE.1993.344061>
- Kipf, A., Kipf, T., Radke, B., Leis, V., Boncz, P., & Kemper, A. (2019). Learned cardinalities: Estimating correlated joins with deep learning. Proceedings of the Conference on Innovative Data Systems Research (CIDR).
- Lai, Z., Wei, Z., Byna, S., Chen, Y., & Agrawal, G. (2021). Adaptive query execution in Apache Spark. IEEE Transactions on Parallel and Distributed Systems, 32(11), 2817-2830.
- Leis, V., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., & Neumann, T. (2015). How good are query optimizers, really? Proceedings of the VLDB Endowment, 9(3), 204-215.
- Marcus, R., & Papaemmanouil, O. (2019). Plan-structured deep neural networks for query performance prediction. Proceedings of the VLDB Endowment, 12(11), 1733-1746.

PostgreSQL Global Development Group. (s. f.). ANALYZE: collect statistics about a database. En PostgreSQL documentation. Recuperado el 25 de enero de 2026, de <https://www.postgresql.org/docs/current/sql-analyze.html>

PostgreSQL Global Development Group. (s. f.). EXPLAIN: show the execution plan of a statement. En PostgreSQL documentation. Recuperado el 25 de enero de 2026, de <https://www.postgresql.org/docs/current/sql-explain.html>

Rajkumar, N., Li, R., & Bahdanau, D. (2022). Evaluating the text-to-SQL capabilities of large language models. arXiv preprint arXiv:2204.00498.

Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A., & Price, T. G. (1979). Access path selection in a relational database management system. Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, 23-34.

VML. (s. f.). Spain | VML. Recuperado el 25 de enero de 2026, de <https://www.vml.com/es/spain>

Wong, A. (2023). What is the difference between TPC-H and TPC-DS benchmarks? Medium. <https://atwong.medium.com/what-is-the-difference-between-tpc-h-and-tpc-ds-benchmarks-cb92fc104c32>

Anexos (Opcional)

Se recogen a continuación los materiales complementarios de la memoria. El código completo del proyecto, incluidos los scripts de carga, benchmarking y verificación, se encuentra en el repositorio indicado en el Anexo A.

<https://github.com/GCardaba/memoria-practicas-1>

Anexo B. Script de conversión a Parquet (ConvertToParquet.scala)

Script en Scala empleado para convertir los ficheros .tbl generados por TPC-H a formato Parquet, optimizado para su lectura desde Apache Spark, tal como se menciona en la metodología.

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.types._

object ConvertToParquet {
  def main(args: Array[String]): Unit = {

    println(" Conversión TPC-H → Parquet (Scala)")
    println("=" * 50)
    println()

    // Configurar Spark
    val spark = SparkSession.builder()
      .appName("TPC-H to Parquet")
      .master("local[*]")
      .config("spark.driver.memory", "4g")
      .config("spark.executor.memory", "4g")
      .config("spark.sql.shuffle.partitions", "8")
      .config("spark.default.parallelism", "8")
      .getOrCreate()

    spark.sparkContext.setLogLevel("WARN")

    // Definir schemas
    val regionSchema = StructType(Array(
      StructField("r_regionkey", IntegerType, nullable = false),
      StructField("r_name", StringType, nullable = false),
      StructField("r_comment", StringType, nullable = true),
      StructField("skip", StringType, nullable = true)
    ))

    val nationSchema = StructType(Array(
      StructField("n_nationkey", IntegerType, nullable = false),
      StructField("n_name", StringType, nullable = false),
      StructField("n_regionkey", IntegerType, nullable = false),
      StructField("n_comment", StringType, nullable = true),
      StructField("skip", StringType, nullable = true)
    ))

    val customerSchema = StructType(Array(
      StructField("c_custkey", IntegerType, nullable = false),
```

```

    StructField("c_name", StringType, nullable = false),
    StructField("c_address", StringType, nullable = false),
    StructField("c_nationkey", IntegerType, nullable = false),
    StructField("c_phone", StringType, nullable = false),
    StructField("c_acctbal", DecimalType(15,2), nullable = false),
    StructField("c_mktsegment", StringType, nullable = false),
    StructField("c_comment", StringType, nullable = false),
    StructField("skip", StringType, nullable = true)
  ))

val supplierSchema = StructType(Array(
  StructField("s_suppkey", IntegerType, nullable = false),
  StructField("s_name", StringType, nullable = false),
  StructField("s_address", StringType, nullable = false),
  StructField("s_nationkey", IntegerType, nullable = false),
  StructField("s_phone", StringType, nullable = false),
  StructField("s_acctbal", DecimalType(15,2), nullable = false),
  StructField("s_comment", StringType, nullable = false),
  StructField("skip", StringType, nullable = true)
))

val partSchema = StructType(Array(
  StructField("p_partkey", IntegerType, nullable = false),
  StructField("p_name", StringType, nullable = false),
  StructField("p_mfgr", StringType, nullable = false),
  StructField("p_brand", StringType, nullable = false),
  StructField("p_type", StringType, nullable = false),
  StructField("p_size", IntegerType, nullable = false),
  StructField("p_container", StringType, nullable = false),
  StructField("p_retailprice", DecimalType(15,2), nullable = false),
  StructField("p_comment", StringType, nullable = false),
  StructField("skip", StringType, nullable = true)
))

val partsuppSchema = StructType(Array(
  StructField("ps_partkey", IntegerType, nullable = false),
  StructField("ps_suppkey", IntegerType, nullable = false),
  StructField("ps_availqty", IntegerType, nullable = false),
  StructField("ps_supplycost", DecimalType(15,2), nullable = false),
  StructField("ps_comment", StringType, nullable = false),
  StructField("skip", StringType, nullable = true)
))

val ordersSchema = StructType(Array(
  StructField("o_orderkey", IntegerType, nullable = false),
  StructField("o_custkey", IntegerType, nullable = false),
  StructField("o_orderstatus", StringType, nullable = false),
  StructField("o_totalprice", DecimalType(15,2), nullable = false),
  StructField("o_orderdate", DateType, nullable = false),
  StructField("o_orderpriority", StringType, nullable = false),
  StructField("o_clerk", StringType, nullable = false),
  StructField("o_shippriority", IntegerType, nullable = false),
  StructField("o_comment", StringType, nullable = false),
  StructField("skip", StringType, nullable = true)
))

val lineitemSchema = StructType(Array(
  StructField("l_orderkey", IntegerType, nullable = false),

```

```

    StructField("l_partkey", IntegerType, nullable = false),
    StructField("l_suppkey", IntegerType, nullable = false),
    StructField("l_linenum", IntegerType, nullable = false),
    StructField("l_quantity", DecimalType(15,2), nullable = false),
    StructField("l_extendedprice", DecimalType(15,2), nullable = false),
    StructField("l_discount", DecimalType(15,2), nullable = false),
    StructField("l_tax", DecimalType(15,2), nullable = false),
    StructField("l_returnflag", StringType, nullable = false),
    StructField("l_linestatus", StringType, nullable = false),
    StructField("l_shipdate", DateType, nullable = false),
    StructField("l_commitdate", DateType, nullable = false),
    StructField("l_receiptdate", DateType, nullable = false),
    StructField("l_shipinstruct", StringType, nullable = false),
    StructField("l_shipmode", StringType, nullable = false),
    StructField("l_comment", StringType, nullable = false),
    StructField("skip", StringType, nullable = true)
  ))

// Mapa de tablas y schemas
val tables = Map(
  "region" -> regionSchema,
  "nation" -> nationSchema,
  "supplier" -> supplierSchema,
  "customer" -> customerSchema,
  "part" -> partSchema,
  "partsupp" -> partsuppSchema,
  "orders" -> ordersSchema,
  "lineitem" -> lineitemSchema
)

val outputDir = "parquet_data"

// Procesar cada tabla
tables.foreach { case (tableName, schema) =>
  print(s"📁 $tableName... ")

  val startTime = System.currentTimeMillis()

  // Leer archivo .tbl
  val df = spark.read
    .option("delimiter", "|")
    .option("mode", "DROPMALFORMED")
    .schema(schema)
    .csv(s"$tableName.tbl")
    .drop("skip")

  // Cache para contar
  df.cache()
  val count = df.count()

  // Guardar como Parquet
  val outputPath = s"$outputDir/$tableName"

  tableName match {
    case "lineitem" => df.repartition(8).write.mode("overwrite").parquet(outputPath)
    case "orders" | "partsupp" => df.repartition(4).write.mode("overwrite").parquet(outputPath)
    case _ => df.write.mode("overwrite").parquet(outputPath)
  }
}

```

```
}

df.unpersist()

val elapsed = (System.currentTimeMillis() - startTime) / 1000.0

println(f"✔ ${count}%,d filas (${elapsed}%.1fs)")
}

println()
println("=" * 50)
println("✔ Conversión completada!")
println()
println(s"📁 Datos Parquet en: ./$outputDir/")

spark.stop()
}
}

// Llamar a main si se ejecuta como script
ConvertToParquet.main(Array())
```